

Cuprins

1	Introducere	3
1.1	Motivarea alegerii temei lucrării	3
1.2	Obiectivele și contribuțiile lucrării	3
2	Fundamente Teoretice	4
2.1	OLTP vs OLAP	4
2.2	Sisteme de baze de date distribuite	4
2.2.1	Arhitecturi de partajare a resurselor	5
2.2.2	Teorema CAP și modelul PACELC	7
2.2.3	Consistență distribuită	9
2.2.4	Strategii de replicare	11
2.2.5	Protocol de consens	12
2.2.6	Optimizarea și planificarea interogărilor	13
2.3	Scalabilitate orizontală și metode de scalare	14
2.3.1	Partiționare orizontală și strategii de distribuție	14
2.3.2	Replicare și toleranță la defecte	14
3	Sisteme Analizate	16
3.1	CockroachDB	16
3.2	TiDB	17
3.3	Citus (Extensie PostgreSQL)	18
3.4	Comparație arhitecturală sintetică	19
4	Metodologie	20
4.1	Utilizarea setului de date TPC-H și caracterizarea workload-ului	20
4.1.1	Schema relațională	21
4.1.2	Scale Factor	21
4.2	Clasificarea interogărilor TPC-H	21
4.3	Infrastructură	22
4.3.1	Specificația hardware și definirea nodurilor	22
4.3.2	Arhitectura rețelei	22

4.3.3	Provizionare și automatizare	23
4.3.4	Instalarea bazei de date	23
4.4	Încărcarea datelor	24
4.4.1	CockroachDB: Import HTTP prin NGiNX	24
4.4.2	TiDB: Compatibil S3 prin MinIO	25
4.4.3	Citus: Transfer direct de fișier	25
4.5	Frameworkul de benchmarking	25
4.5.1	Implementare	25
4.5.2	Managementul conexiunii și izolarea	26
4.5.3	Colectarea rezultatelor din program și exportul	27
4.6	Metodologia de măsurare	27
4.6.1	Warmup runs (rulări inițiale)	27
4.6.2	Rulările de măsurare	27
4.7	Fairness și comparabilitate	28
5	Rezultate obținute	29
5.1	Grupa 1 - Interogări dominate de scanări	29
5.2	Grupa 2 - Interogări dominate de îmbinări	32
5.3	Grupa 3 - Interogări conduse de subinterogări	35
5.4	Grupa 4 - Interogări în mai multe etape	38
5.5	Sinteză intergrupală	41
6	Analiza rezultatelor	43
6.1	CockroachDB	43
6.2	TiDB	44
6.3	Citus	44
7	Concluzii	45
7.1	Direcții viitoare de cercetare	45
A	Schemă și configurație de partiționare	46
A.1	Schema comună TPC-H	46
A.2	Configurația de distribuție Citus	48
B	Tabel mediane	49
C	Mesaje eroare	50
	Bibliografie	52

Capitolul 1

Introducere

1.1 Motivarea alegerii temei lucrării

Creșterea volumului de date a generat o necesitate pentru adoptarea sistemelor distribuite de baze de date SQL, ceea ce a creat o nevoie pentru înțelegerea arhitecturală și a diferențelor între aceste sisteme ce pot duce la diferențe măsurabile în condiții reale. Această lucrare dorește să adreseze această evaluare prin măsurarea a 3 sisteme diferite: CockroachDB, TiDB și Citus folosindu-se de benchmark-ul de suport decizional TPC-H [1] pe clustere de 3, 5, 10 și 20 de noduri, implementate pe un hardware identic. Cele trei sisteme au fost selectate deoarece reprezintă abordări arhitecturale distincte. În cazuri din viața reală un sistem de baze de date deservește mai multe acțiuni, un sistem ce în timpul zilei suportă tranzacții este așteptat să suporte și interogări de tip raport și agregări. Menținerea constantă a infrastructurii împreună cu restul deciziilor arhitecturale garantează că diferențele de performanță observată reflectă diferențele arhitecturale între sisteme și nu diferențe hardware.

1.2 Obiectivele și contribuțiile lucrării

Această lucrare urmărește trei obiective principale: stabilirea unui profil de performanță pentru fiecare sistem pentru benchmark-ul TPC-H, identificarea și explicarea diferențelor arhitecturale ce reies din rezultate și producerea unei metodologii de măsurare ce poate fi extinsă pentru măsurarea oricărui sistem.

Contribuțiile sunt multiple: folosirea unui set de date empirice comparative acoperind trei sisteme SQL distribuite pe patru configurații de cluster, o analiză ce conectează rezultate brute de diferențe arhitecturale de execuție, crearea unei metodologii replicabile implementată în Java, cu infrastructură provizionată prin Terraform.

Capitolul 2

Fundamente Teoretice

Acest capitol prezintă fundamentele teoretice necesare pentru diferențierea sistemelor de baze de date distribuite analizate în lucrare. Analizăm diferența dintre workload-uri tranzacționale și analitice, arhitecturile distribuite și mecanismele lor.

2.1 OLTP vs OLAP

Diferențele dintre procesarea tranzacțională și analitică sunt esențiale pentru evaluarea sistemelor de baze de date distribuite, deoarece fiecare impune cerințe diferite asupra arhitecturii și modului de funcționare.

OLTP (Online Transaction Processing) se caracterizează prin tranzații scurte, cu latență mică, care afectează puține rânduri, prin operații de tip INSERT, UPDATE, DELETE punctuale. Metricile cheie pentru acest sistem sunt: throughput (tranzacții/secundă) și latența la percentilele p95/p99. Exemple pentru sisteme OLTP: sisteme bancare, magazine online, gestiuni de magazie.

OLAP (Online Analytical Processing) se caracterizează prin interogări complexe, de tip read-heavy care scanează volume mari de date prin agregări, GROUP BY pe multiple dimensiuni, JOIN-uri într tabele mari, subinterogări. Exemple pentru sisteme OLAP: rapoarte financiare, analiza vânzărilor.

Diferențele fundamentale între aceste două sisteme acoperă volumul de date procesat per interogare, concurența tranzațiilor (mii de tranzații mici vs zeci de interogări scumpe) și strategiile de optimizare (indexare B-tree vs. Columnar storage, partitioning, materialized views).

Este important de menționat că sistemele moderne încearcă să deservească ambele tipuri de workload, dar cu compromisuri.

2.2 Sisteme de baze de date distribuite

Un sistem de baze de date distribuit stochează și procesează datele pe multiple noduri interconectate într-o rețea, oferind unui client o interfață unificată care ascunde complexitatea distri-

buției.

Motivele principale pentru utilizarea unui sistem distribuit sunt: scalabilitatea orizontală (creșterea capacității de procesare prin adăugarea de noduri), disponibilitatea ridicată (un nod se poate opri din anumite motive, dar serviciul rămâne activ) și distribuția geografică (reducerea latenței pentru utilizatorii unui serviciu global).

Provocările principale includ coordonarea tranzacțiilor, menținerea consistenței datelor și optimizarea interogărilor care au nevoie de mai multe noduri.

2.2.1 Arhitecturi de partajare a resurselor

Sistemele distribuite pot fi clasificate în funcție de modul în care nodurile partajează resursele. Aceste alegeri au implicații directe asupra scalabilității și asupra performanței. Se identifică în anul 1986 de către Michael Stonebraker trei categorii arhitecturale pentru clasificarea împărțirii resurselor și anume: shared-memory, shared-disk și shared-nothing [2]. Această împărțire este consolidată ulterior de către DeWitt și Gray [3] iar ea rămâne valabilă și în ziua de astăzi, chiar dacă implementările sistemelor moderne distribuite pot combina elemente din mai multe paradigme.

Arhitectura **shared-memory** denumită uneori și shared-everything presupune ca toate nodurile de procesare dintr-un sistem distribuit să aibă acces la un fond comun pentru memorie și stocare. În acest sistem mai multe procesoare operează asupra aceluiași sistem de discuri. Avantajul principal constă în simplitatea administrării acestui sistem și simplitatea accesului la date, orice procesor poate accesa orice fragment de date de care are nevoie, fără a fi nevoie de o comunicare prin rețea. Acest lucru este realizat deseori prin mecanisme de locking, pe măsură ce numărul de procesoare crește în acest sistem, accesul poate fi blocat pentru întregul sistem. Scalabilitatea acestui sistem este restricționată la un număr limitat de noduri, iar coruperea datelor de către unul din noduri poate afecta un întreg sistem. Din aceste motive această arhitectură este întâlnită rareori, iar când este, este într-un context în care volumul de date și numărul de noduri rămân moderate.

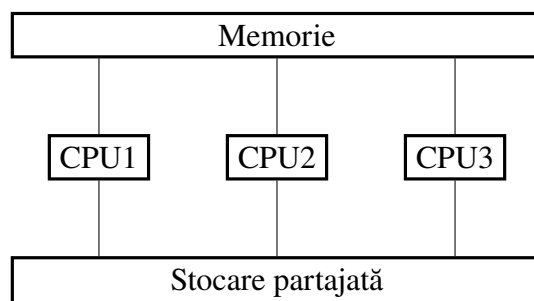


Figura 2.1: Arhitectura shared-memory/shared-everything

Arhitectura **shared-disk** reprezintă o soluție la „mijloc”, fiecare nod dispune de memorie proprie și putere de procesare independentă (propriul procesor), însă toate nodurile accesează

același sistem de stocare. Această separație permite o formă de paralelism la nivelul procesării, fiecare nod putând să execute procese independent. Principalul dezavantaj, ca în sistemul anterior constă în necesitatea unui mecanism de locking, sau blocare a datelor: atunci când mai multe noduri încearcă să modifice aceleași date simultan. Sincronizarea acestor lock-uri între sisteme introduce o supra-sarcină sistemului. Acest cost de sincronizare limitează numărul de noduri pe care le poate avea un sistem shared-disk. Ulterior Stopford observă că, o implementare shared-disk poate simula comportamentul unei arhitecturi shared-nothing prin partiționare logică a datelor, reducând volumul de lock shipping [4].

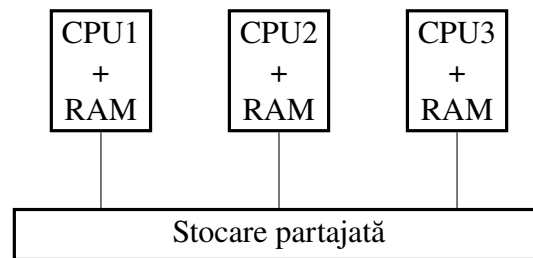


Figura 2.2: Arhitectura shared-disk

Arhitectura **shared-nothing** este deseori argumentată ca alternativa cea mai eficientă din punct de vedere al costurilor [2]. Această implementare elimină complet partajarea oricăror resurse între noduri. Fiecare nod deține propriile resurse: procesor, memorie și stocare. Toată coordonarea între noduri se realizează printr-o rețea [3]. Absența oricărei partajări de resurse permite fiecărui nod să proceseze datele independente, fiecare lucrând doar cu fragmentul de date care îi este atribuit. Faptul că fiecare nod se poate ocupa independent de procesarea unui set de date, permite descompunerea interogărilor complexe (în mod uzual de tip analitic) în mai multe sub-interogări care se execută simultan pe multiple noduri, rezultatele parțiale obținute fiind ulterior agregate pentru obținerea rezultatului final. Scalabilitatea orizontală este nelimitată, de asemenea toleranța la defecte este mare, în cazul defectării unui nod, nu se afectează funcționarea celorlalte, iar datele de pe acel nod pot fi redistribuite și replicate pentru a asigura disponibilitate.

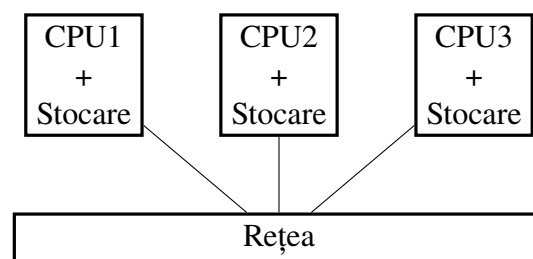


Figura 2.3: Arhitectura shared-nothing

Dezavantajul principal al arhitecturii shared-nothing este reprezentat de complexitatea introdusă de partiționarea datelor și gestionarea interogărilor pe noduri multiple. Operațiile ce afectează mai multe noduri (agregări globale, join-uri distribuite, etc.) implică un transfer de date între noduri prin rețea. Performanța operațiunilor între mai multe noduri depinde critic de

strategiile de partiționare. Stonebraker recunoaște aceste dificultăți dar argumentează că sistemele de echilibrare a bazelor de date automate pot atenua semnificativ aceste dificultăți [2].

Toate sistemele analizate în această lucrare, CockroachDB, TiDB și Citus adoptă diferite variante ale arhitecturii shared-nothing, astfel putem observa consensul industriei moderne asupra modelului folosit.

2.2.2 Teorema CAP și modelul PACELC

Pentru a analiza sistemele de baze distribuite, avem nevoie de un cadru teoretic care să motiveze compromisurile arhitecturale adoptate de către fiecare sistem. În această secțiune examinăm concepte din domeniul sistemelor distribuite precum: teorema CAP și modelul PACELC, algoritmi de consens care asigură coordonarea între noduri, strategii de partiționare a datelor și mecanismele de scalabilitate ce permit creșterea capacității unui cluster (un ansamblu de noduri interconectate, ce funcționează ca un sistem unitar). Aceste concepte oferă fundamentele pentru evaluarea comparativă a sistemelor de baze de date: CockroachDB, TiDB și Citus din capitolele ulterioare.

Formularea și formalizarea teoremei CAP

Teorema CAP este un fundament al sistemelor de baze de date distribuite, stabilind bazele limitărilor și garanțiilor acestor sisteme. Abrevierea CAP vine de la consistency (consistența datelor), availability (disponibilitatea serviciului) și tolerance to network partitions (toleranța la partiționarea datelor). Teorema a fost lansată inițial de către Eric Brewer în cadrul unei prezentări la o conferință ACM din anul 2000, enunțul teoremei afirmă că un sistem distribuit nu poate garanta simultan decât 2 din toate cele 3 proprietăți [5]. Doi ani mai târziu teorema a primit o demonstrație formală din partea lui Gilbert și Lynch [6]

Teorema implică 3 strategii posibile, fiecare renunțând la o proprietate în detrimentul celorlalte două. Prima opțiune este renunțarea la toleranța la partiții, comunicarea între noduri este întreruptă. Brewer [5] menționează mecanisme precum protocolul two-phase commit (comiterea datelor în 2 faze), un protocol ce presupune blocarea resurselor, nodurile participante decid „da” sau „nu”. Faza a doua este strict faza de comitere a modificărilor.

A doua opțiune este renunțarea la proprietatea de disponibilitate. Datele pot fi utilizate doar atunci când consistența lor este garantată. Mecanismul menționat pentru această opțiune de Brewer [5] este pesimistic locking (blocarea pesimistă), caz în care modificarea unui obiect rămâne blocată până la propagarea modificării către toate nodurile.

A treia opțiune este renunțarea la proprietatea de consistență în detrimentul proprietății de disponibilitate și cel de partiționare. Această strategie implică utilizarea mecanismelor de blocare optimistă și protocoalelor de rezolvare a conflictelor, care reconciliază stările divergente.

Dintre cele trei opțiuni, renunțarea la toleranța la partiții, CA (consistență & disponibilitate) nu este posibilă în sistemele distribuite care întâlnesc probleme de conectivitate. Orice sistem

distribuit ce are noduri conectate la o rețea va experimenta, la un moment dat, întârzieri și/sau pierderi de pachete. Acest fapt reduce decizia la cele 2 sisteme, CP (consistență și partiționare) și AP (disponibilitate și partiționare). Brewer [7] revine în anul 2012 și subliniază că decizia nu este una binară, ea este de fapt un spectru, un sistem distribuit poate combina niveluri diferite de consistență și disponibilitate în funcție de nevoi și de context.

Critica teoremei CAP

O analiză critică asupra teoremei CAP a fost realizată și de către Klepmann[8] care identifică probleme fundamentale în definiția lui Brewer și formalizarea ei de către Gilbert și Lynch. O problemă centrală identificată: termenii teoremei nu au constrângeri clare, termenii definiții sunt înguști. Consistența în contextul CAP se referă exclusiv la „linearizability”, garanția că sistemul se comportă ca și cum ar exista o singură copie a datelor, și nu la consistența din modelul ACID, argumentând că în contextul sistemelor distribuite, consistența se referă la un spectru de modele cu varii puncte forte și slabe precum *casual consistency*, *sequential consistency* ori *PRAM*.

Similar, definiția CAP impune disponibilitate, dar nu definește o limită în care un sistem disponibil să răspundă, astfel în definiția inițială un sistem care răspunde într-o săptămână poate fi considerat disponibil, un lucru neacceptat în industrie spune Klepmann [8].

Ca alternativă Klepmann propune un nou sistem, numit delay-sensitivity (sensibilitatea la întârzieri), care propune analizarea variațiilor de latență a operațiunilor atunci când rețeaua devine lentă sau inaccesibilă. Această nouă definiție permite clasificarea operațiunilor în două categorii: sensibilă la întârziere, dacă latența crește proporțional cu problemele din rețea sau independentă, dacă latența rămâne constantă indiferent de condițiile rețelei. Klepmann [8] arată că **linearizability** (orice operație pare că se execută instantaneu, la un moment în timp, iar toate nodurile văd același lucru în același timp) necesită această dependență pentru citiri și scrieri. **Sequential consistency** (garanția că orice operație se execută într-o ordine globală, dar nu neapărat în timp real), în acest sistem fiecare sistem vede operațiile în ordinea corectă dar nu există o sincronizare cu un ceas, permite citiri rapide dar scrieri pasibile de întârzieri. **Casual consistency** (garanția că operațiile legate printr-o relație cauză-efect sunt observate în ordinea corectă) poate funcționa complete neafectată de întârzierile din rețea.

	latență scriere	latență citire
linearizability	$O(d)$	$O(d)$
sequential consistency	$O(d)$	$O(1)$
casual consistency	$O(1)$	$O(1)$

Tabela 2.1: Scalare modele de consistență

Teorema PACELC

În anul 2012, Abadi [9] susține că ignorarea compromisului dintre latență și consistență este o omisiune majoră. La fel ca în teorema CAP, teorema PACELC susține că într-un sistem

distribuit, în cazul partiționării, trebuie să facem un compromis și să alegem între disponibilitate și consistență. Teorema introduce noțiunea de latență în formulare și susține că în timpul unei partiționări putem alege între A (availability, disponibilitate) și C (consistență) sau E (else), putem alege între L (latență) și C (consistență).

Teorema PACELC prezintă 4 configurări și anume: **PA/EL**, sistemele prioritizează disponibilitatea și latența joasă, de exemplu sisteme precum Apache Cassandra. **PC/EC**, sistemele prioritizează consistența în detrimentul latenței mărite sau a disponibilității partiționării, de exemplu CockroachDB. **PA/EC**, sistemele optimizează pentru disponibilitate în momentul partiționării dar pentru consistență în modul normal de operare. **PC/EL**, sistemele aleg să prioritizeze consistența în partiționare, dar latență joasă în restul timpului de operare.

2.2.3 Consistență distribuită

Anterior am văzut cum teorema CAP și extensia ei, teorema PACELC stabilesc că un sistem distribuit operează pe un spectru, cu o balanță între consistență, disponibilitate și toleranța la partiționare. Aceste modele sunt teoretice, în practică diferite sisteme au nevoie de diferite modele de consistență.

Modele de consistență

Modelele de consistență pot fi aranjate pe un spectru, de la cele mai puternice, modele care simulează comportamentul sistemelor pe un singur nod, până la cele mai slabe care pot garanta consistența între noduri în timp. Fiecare sistem își alege anumite compromisuri între corectitudine, latență și disponibilitate după cum am văzut în extensia PACELC [9].

Linearizability Termenul de Linearizability, referit și ca atomic consistency (consistență atomic - atomic în acest context se referă la operațiune acționată cu rezultat totul sau nimic) este unul dintre cele mai puternice modele de consistență pentru operațiuni asupra unui singur obiect [10]. Fiecare operațiune se execută instantaneu într-un singur moment în timp aflat între momentul invocării și răspuns. Două proprietăți se dezvoltă din aceasta definiție: *local*: un sistem compus din obiecte individuale linearizabile este în sine linearizable, această proprietate dând voie unui sistem să fie modular [10]. Cea de a doua proprietate *nonblocking* (non-blocant), un proces care invocă o operație nu este obligat să aștepte după un sistem suspendat să completeze. În cazul teoriei CAP, linearizability plasează un sistem în categoria CP (consistență și partiționare), menținând această consistență într-un cluster suntem obligați să refuzăm anumite cereri care nu întrunesc cvorumul [6]. În modelul PACELC, sistemele ce implementează modelul linearizability sunt clasificate ca PC/EC, se plătește un cost asupra latenței în operațiunile normale [9].

Sequential consistency Sequential consistency, consistența secvențială este un model mai slab ca linearizabilitatea. Acest model impune ca execuția să fie echivalentă cu o anumită istorie secvențială care păstrează ordinea de execuție pentru fiecare proces individual, fără a impune păstrarea ordonării globale în timp real între mai multe procese [10]. O consecință a acestei relaxări a constrângerii de timp real este că consistența nu este o proprietate locală, un sistem compus din mai multe obiecte consistente secvențial pot eșua în a fi consistent secvențial în ansamblu [10]. Această proprietate are implicații are consecințe în proiectarea sistemelor deoarece compozabilitatea nu poate fi presupusă. În schimb această relaxare permite latențe mai mici operațiilor de citire în anumite configurații, făcând acest model de consistență o alegere potrivită în sistemele în care sincronizarea globală la fiecare operație de citire ar fi costisitoare.

Causal consistency Causal consistency, consistența cauzală ocupă o poziție intermediară pe spectrul modelelor de consistență, acest model oferă garanții semnificative de siguranță fără a fura din costul complet de latență al unui sistem consistent puternic. Modelul se bazează pe formalizarea ordonării cauzale, propusă de Lamport și anume: dacă operația A precedă operațiunea B (notat $A \rightarrow B$), atunci toate procesele trebuie să observe A înaintea lui B [11]. O caracteristică definitorie a acestui model de consistență este că este unul din cele mai puternice modele care permite latențele operațiilor să rămână independente de latența rețelei.

Session guarantees Între consistența cauzală și cea eventuală avem modelul care garantează garanții centrate pe client, denumit și garanții de sesiune, ce oferă proprietăți de siguranță la nivelul de client fără a necesita coordonare globală [12]. Trei importante garanții sunt deosebit de relevante: **Read-your-writes** (citirea propriilor scrieri) asigură că un proces observă întotdeauna propriilor sale scrieri anterioare la citirile ulterioare și nu va întâlni niciodată o versiune învechită a datelor pe care el însuși le actualizează, **Monotonic reads** (citiri monotone) garantează că, odată ce un proces a observat o anumită versiune a unui obiect, nici o citire ulterioară a acelui obiect nu va returna o versiune mai veche, **Monotonic writes** (scrieri monotone), asigură că scrierile emise de un proces sunt serializate în ordinea în care au fost înregistrate. Aceste trei garanții sunt implementate frecvent prin rutarea cererilor unui client în mod consecvent către aceeași replică, și reprezintă un compromis larg în sistemele eventual consistente [12].

Eventual consistency Consistența eventuală este considerat unul din cele mai slabe modele de consistență și este folosit în general în sistemele cu disponibilitate critică. Este definit astfel: într-o execuție infinită, dacă toate scrierile încetează și are loc o comunicare suficientă între noduri, toate replicile vor converge către aceeași valoare [12]. Acest model nu impune nici o limită de timp pentru convergență, nici nu se impun constrângeri asupra valorilor ce sunt citite în timpul ferestrei de convergență. Fereastra de inconsistență (momentul de convergere pentru replicare, decalajul de replicare), diferite replici pot returna valori diferite pentru același obiect, această consecință este deseori atenuată prin garanțiile de sesiune [12].

2.2.4 Strategii de replicare

Modelul de consistență adoptat de un sistem produce efecte asupra strategiei de replicate utilizate de acesta. Replicarea are două obiective: să îmbunătățească viteza și accesibilitatea citirilor prin distribuirea acestora pe mai multe read-replicas (noduri replicate doar pentru citire) și îmbunătățirea toleranței la defecte și erori, asigurând accesibilitatea datelor chiar și dacă anumite noduri sunt supuse unor erori devenind nedisponibile. Metoda în care aceste scrieri sunt distribuite pe toate nodurile replică ne dă garanțiile de consistență pe care clienții care accesează aceste date le au [13].

Sincronă și Asincronă

Replicarea propagă actualizările datelor între mai multe noduri ale unui sistem distribuit cu scopul de a asigura toleranța la defecte, disponibilitatea și scalabilitatea operațiilor de citire. În replicarea sincronă, o operație de scriere este confirmată clientului doar după ce fiecare replică desemnată a comis durabil actualizarea, garantând astfel o consistență tare cu prețul unei latențe mai mari și al unei disponibilități reduse atunci când o replică devine inaccesibilă [12]. În replicarea asincronă, nodul principal confirmă clientul imediat și diseminează actualizarea către replici în fundal, ceea ce îmbunătățește debitul și tolerează căderile replicilor, însă introduce o fereastră de divergență cunoscută drept întârziere de replicare, conducând la o consistență eventuală [12].

Lider unic

Replicarea cu lider unic (lider - urmăritor) desemnează un singur nod drept lider, care serializează toate operațiile de scriere și propagă un registru al modificărilor către toți urmăritorii săi, care deserveșc traficul de citire. Această arhitectură simplifică prevenirea conflictelor deoarece scrierile sunt ordonate liniar într-un singur punct, dar introduce un blocaj unic și singur, în cazul unei erori în nodul lider, de aceea trebuie implementat un sistem de alegere automată a liderului, implementată frecvent prin protocoale de consens precum Raft sau Multi-Paxos [14].

Lideri multipli

Replicarea cu lideri multipli (activ - activ) permite ca operațiile de scriere să fie acceptate simultan de mai multe noduri-lider, fiecare propagând ulterior modificările către ceilalți lideri și către urmăritorii locali. Principala dificultate o constituie însă rezolvarea conflictelor de scriere concurentă, care poate necesita politici deterministe de tipul „ultimul scriitor câștigă”, vectori de versiuni sau o logică de fuziune la nivelul aplicației [13].

2.2.5 Protocol de consens

Anterior am discutat despre cum strategiile de replicare definesc cum datele ajung pe toate nodurile, protocolul de consens definește felul în care nodurile cad de comun acord că datele copiate sunt corecte și cum se repară replicările în cazul erorilor. Problema consensului necesită ca toate nodurile/procese să fie de acord asupra unei singuri valori, chiar și dacă anumite noduri nu au replicat acea valoare sau au dat erori, acest proces se definește prin următoarele proprietăți: **agreement** - două procese care nu sunt în eroare nu pot decide diferit unul față de celălalt, **validity** - valoarea agreată este aleasă de oricare proces care nu este în eroare și **termination** - orice proces care nu este în eroare dă un răspuns într-un final [15]. Consensul este mecanismul prin care replicările bazate pe cvorum realizează modelul de linearizability descris anterior, asigurând că toate replicările fără erori primesc și aplică în aceeași secvență operații, asigurând astfel un comportament care simulează logica unui sistem pe un singur nod [10].

Abordarea automatelor de stare (state machines)

O metodă de implementare a serviciilor tolerante la defecțiuni în sistemele distribuite este prin metoda automatelor de stare, formalizată de Schneider [16]. Un serviciu este modelat ca un automat de stare determinist, alcătuit din variabile de stare și din comenzi ce le transformă. Prin replicarea acestui automat pe mai multe procese și prin asigurarea faptului că fiecare replică ne-defectă execută aceeași secvență de comenzi, ansamblul manifestă comportamentul unui server unic, cu disponibilitate ridicată.

Paxos și multi-Paxos

Paxos, introdus de Lamport [17] și ulterior reformulat într-o prezentare accesibilă [18]. Acesta rezolvă problema consensului într-un sistem asincron, predispus la defecțiuni de tip crash și bazat pe schimb de mesaje. Paxos alege o valoare prin două faze conduse de trei roluri: propunători, acceptanți și învățători: o fază de pregătire, ce stabilește un număr de buletin și colectează promisiunile anterioare, urmată de o fază de acceptare, în care un cvorum de acceptanți votează valoarea. Multi-Paxos amortizează faza de pregătire prin alegerea unui lider stabil care propune o secvență de valori, implementând astfel abstracția jurnalului replicat, esențială în replicarea automatelor de stare; această variantă stă la baza serviciului Chubby al Google [19].

Raft

Raft, prezentat de Ongaro și Ousterhout [14], este un algoritm de consens proiectat explicit pentru inteligibilitate, fiind totodată echivalent cu Multi-Paxos sub aspectul toleranței la defecțiuni și al performanței. Autorii descompun consensul în trei subprobleme relativ independente:

alegerea liderului, replicarea jurnalului și siguranța. Nodurile funcționează în una dintre trei stări: follower (adept), candidat sau lider.

Paxos vs Raft

Howard și Mortier [20] întreprind o comparație sistematică, reformulând Multi-Paxos în terminologia Raft și concluzionează că cei doi algoritmi sunt structural mult mai apropiați decât sugerează în mod tradițional literatura, diferind în principal prin abordarea alegerii liderului. Raft impune o restricție de alegere care permite doar nodurilor cu jurnale suficient de actuale să devină lideri, evitând astfel schimbul post-alegere de jurnal pe care Paxos îl impune atunci când un lider proaspăt ales deține un jurnal incomplet. Această decizie de proiectare face ca alegerea liderului în Raft să fie surprinzător de eficientă, în vreme ce Paxos păstrează o flexibilitate mai mare cu privire la replica ce poate prelua conducerea. Autorii susțin că ambele protocoale implementează în cele din urmă paradigma automatelor de stare a lui Schneider [16].

2.2.6 Optimizarea și planificarea interogărilor

Optimizarea bazată pe cost

Optimizarea interogărilor din bazele de date relaționale se bazează aproape universal pe paradigma bazată pe cost, care separă declarația SQL de calea de acces procedurală efectivă și selectează dintre planurile algebric echivalente, pe cel cu cost estimat minim [21]. Tehnica se bazează pe trei componente interdependente și anume: un mecanism de enumerare, ce exploatează spațiul planurilor, un set de formule de selectivitate bazate pe statistici, și o funcție de cost care combină costul CPU (procesor, putere de calcul) și I/O (input - output) [22, 23]. Cea mai mare problemă rămâne însă estimarea cardinalității, un studiu empiric sistematic realizat pe Join Order Benchmark a arătat că estimatoarele de nivel industrial produc în mod sistematic erori de un factor de 1000 sau mai mult în cazul interogărilor cu multiple join-uri, iar modelul de cost are o influență substanțial mai mică asupra performanței decât aceste estimări [24].

Planificarea interogărilor distribuite - etape și provocări

Extinderea modelului cost-based într-un context distribuit introduce o dimensiune calitativ nouă: pe lângă deciziile privind metoda de acces local, optimizatorul trebuie să determine unde se execută fiecare operator și cum se deplasează rezultatele intermediare între situri, întrucât transferul prin rețea domină de regulă funcția de cost [25]. Descompunerea canonică organizează compilarea în patru etape: parsarea, rescrierea logică, localizarea datelor pe fragmentele partiționate și optimizarea globală, care întreține ordonarea join-urilor cu selecția sitului și strategia de transfer [26].

Pushdown pentru predicate și funcții de agregare

Rescrierile de tip pushdown (evaluarea în jos, cât mai aproape de date a predicatelor - filtrelor și a funcțiilor de agregare), aplicate atât selecțiilor cât și agregatelor decompozabile, constituie familia de optimizări cu cel mai bun raport cost-beneficiu într-un motor de execuție distribuit, întrucât evaluarea operatorilor în apropierea datelor reduce simultan costul de I/O și volumul de octeți transferați prin rețea [27, 28]. Atunci când o funcție agregat este decompozabilă, posibilă să fie evaluată parțial (COUNT, SUM, MIN, MAX), optimizatorul introduce o agregare parțială deasupra fiecărei operații de scan și o agregare finală deasupra shuffle-ului, reducând semnificativ cardinalitatea transferată între noduri [22, 29].

2.3 Scalabilitate orizontală și metode de scalare

Scalabilitatea orizontală reprezintă capacitatea unui sistem distribuit de suporta creșteri în volumul de calcul prin adăugarea de noduri, aceasta este în contrast cu scalarea pe verticală care se face prin creșterea puterii de calcul pe nodurile deja existente. Scalarea pe verticală este limitată de resursele unei singuri mașini de calcul. În contextul analitic scalarea orizontală adresează majoritatea problemelor bazelor de date, precum creșterea volumului de date și cerințele de interogare. Impedimentul principal este costul de coordonare între noduri, care crește pe măsura introducerii de noi noduri [26].

2.3.1 Partiționare orizontală și strategii de distribuție

Partiționarea orizontală, cunoscută în domeniu drept „sharding” reprezintă o descompunere a unei tabele mari în bucăți mai mici, fiecare bucată fiind stocată pe un server diferit. O cheie de partiționare determină care bucată primește care rând de date [26]. Trei strategii principale de partiționare sunt aplicate în ziua de astăzi și anume: **partiționarea prin hash**, care aplică o funcție hash uniformă cheii pentru a echilibra sarcina, în detrimentul costului de explorare a intervalelor, **partiționarea prin interval**, care păstrează ordonarea cheii și favorizează interogările pe intervale, dar este predispusă unei deformări în cazul distribuirii neuniforme ale cheii și **partiționarea prin listă** care este folosită când domeniul cheii de distribuire este mică. DeWitt și Gray au demonstrat, în analiza lor asupra bazelor de date paralele, că strategia de partiționare are un impact direct în paralelismul realizabil [3].

2.3.2 Replicare și toleranță la defecte

Replicarea reprezintă procesul de menținere a mai multor copii ale datelor partiționate pe mai multe noduri distincte, astfel obținând atât durabilitate împotriva defecțiunilor hardware, cât și scalabilitate de citire [30]. Topologiile de replicare sunt clasificate în 3 tipuri și anume:

single-leader (un singur lider) unde un nod primește toate scrierile apoi le distribuie, **multi-leader** (cu mai mulți lideri) unde mai multe servere pot primi scrieri simultan (o implementară mai complicată, dar mai rapidă) și **leaderless** (fără lider) unde nici un server nu conduce, toate sunt egale, alegerea unei topologii de replicare determină profilul de latență, cerințele pentru rezolvarea conflictelor și garanțiile de consistență în caz de partiționare a rețelei [30]. Sistemele actuale distribuite tind să aleagă între protocoalele de consens Raft și Paxos, care asigură scrieri validate ce supraviețuiesc defecțiunilor apărute pe nodurile minorități prin cerința de cvorum, exprimată obișnuit ca $W + R > N$, unde W - numărul de noduri care trebuie să confirme o scriere, R - numărul de noduri care trebuie să citească pentru a obține date fiabile și N - numărul total de replici.

Capitolul 3

Sisteme Analizate

Acest capitol urmărește să analizeze arhitectura internă a celor 3 sisteme de baze de date SQL distribuite ce le vom analiza cu obiectivul de a stabili fundamentul arhitectural în raport cu care vor fi interpretate rezultatele TPC-H din capitolele următoare. Cele trei sisteme se poziționează diferit în relație cu decuplarea dintre stratul SQL de procesare și stratul de stocare dar și prin granularitatea în care se gestionează replicarea și consensul. CockroachDB integrează stratul SQL peste un strat de stocare KV (chei - valoare) distribuit încorporat. TiDB are o abordare diferită, dezbină clar procesarea, stocarea și metadatele în trei niveluri scalabile independent. Citus extinde un server PostgreSQL clasic, cu o topologie coordonator-worker, realizată prin extensii și interceptarea planificatorului și executorului [31, 32, 33]. Orice decizie la acest nivel are implicații directe observate într-un workload analitic.

3.1 CockroachDB

Arhitectura peer-to-peer simetrică, distribuția rangurilor CRDB este conceput ca un cluster simetric de tip peer-to-peer în care fiecare nod execută un binar identic ce expune atât stratul SQL, cât și stratul KV, eliminând o distincție dintre rolul de master și worker [31, 34]. Spațiul de chei este organizat ca o singură hartă și este subdivizat în segmente contigue (adiacente). Varianta inițială SIGMOD specifică o dimensiune aproximativă de 64MiB varianta actuală (configurabilă prin variabila `range_max_bytes`) este implicit la valoarea de 512 MiB [31, 34]. Fiecare segment este replicat printr-un grup independent de consens ce implementează protocolul Raft. Fiecare rang este replicat cu trei replici distribuite pe domenii de defecțiune distince, o singură replică per rang deține un contract de rang (range lease) și este întodeauna co-localizată cu liderul Raft. Acest aranjament permite citirile consistente la nivel local, fără un schimb suplimentar de consens [34]. Plasarea, divizarea și reechilibrarea rangurilor sunt coordonate printr-un protocol de tip gossip și un alocator automat, și nu printr-un planificator central

Mecanismul de query optimization Optimizarea interogărilor în CRDB este realizată de un optimizator bazat pe costuri [35]. Optimizatorul se folosește de reguli de transformare organi-

zate în faze de normalizare și explorare. Estimarea costurilor se bazează pe statistici care includ numărul de rânduri, numărul de valori distincte și nule, precum și histogramme de adâncime egală menținute per coloană de index, cu actualizare automată declanșată atunci când un procent din rândurile unui tabel au fost modificate [35]. După optimizarea logică, o etapă separată de planificare fizică produce un graf aciclic orientat de procesoare conectate prin fluxuri tipizate; acest plan DistSQL determină care noduri participă la execuție pe baza localității rangurilor, cu obiectivul explicit de a împinge procesarea cât mai aproape de datele pe care le consumă [36].

Strategii de join & agregare Modelul de execuție distribuită al CRDB suportă patru strategii de join, selectate de optimizator în funcție de dimensiunea datelor, disponibilitatea indecșilor și proprietățile de ordonare ale intrărilor [37]. Join-urile hash constituie mecanismul implicit, construind o tabelă hash în memorie pe intrarea mai mică și deversând pe disc atunci când memoria impune acest lucru. Joinurile prin interclasare (merge joins) sunt preferate atunci când ambele relații sunt citite într-o ordine de sortare compatibilă prin indecși pe coloanele de join și sunt utilizate exclusiv în planuri distribuite [37]. Join-urile prin căutare (lookup joins) vizează cazul unei asimtetrii pronunțate de dimensiune între cele două părți. Join-urile de tip apply (apply joins) servesc ca ultima instanță pentru subinterogările corelate care nu pot fi decorelate și sunt semnalate ca un tipar de evitat.

Agregarea urmează un tipar în două etape, un procesor local calculează o agregare parțială per rang, după care un router cu partiționare hash redirecționează rezultatele parțiale către un agregator final care le combină [36].

Partiționare și colocation Partiționarea explicită a tabelelor în CRDB este implementată ca o funcționalitate enterprise care acceptă specificații de partiționare prin listă sau interval asupra coloanelor oricărui index și se integrează cu configurațiile de zonă pentru a varia setările de replicare și constrângerile de plasare la granularitatea partiției [34].

3.2 TiDB

Arhitectura separata TiDB este compus din trei niveluri distincte din punct de vedere arhitectural: un nivel de procesare SQL fără state, un nivel de stocare orientat pe rânduri replicat prin Raft și un serviciu de metadate. Acest aranjament arhitectural permite scalarea independentă a fiecărui nivel față de celelalte [32]. Nivelul de procesare este format din procese TiDB care implementează protocolul MySQL, parsează, planifică interogările și coordonează execuția distribuită. Deoarece aceste noduri nu au state, ele pot fi scalate orizontal fără mișcări de date. Mecanismul de stocare, TiKV expune o hartă KV ordonată în care fiecare rând este codificat într-o cheie compusă din identificatorul tabelului și identificatorul rândului. Spațiul de chei este partițional în segmente denumite regiuni iar fiecare regiune constituie un grup Raft independent al cărui lider gestionează toate cererile de citire și scriere pentru acel segment [32]. În final,

nivelul de metadata PD (placement driver) este un cluster de dimensiuni reduse, care folosește stocare KV (prin etcd) care planifică mutările pentru echilibrarea sarcinii [32]. Separarea clară dintre calcul și stocare este o caracteristică definitorie pentru TiDB.

Mecanismul de optimizare a interogărilor Optimizarea interogărilor în TiDB se execută în 2 faze, prima etapă este o etapă bazată pe reguli ce produce un plan logic normalizat, urmat de o etapă de planificare bazată pe cost [32]. Planificarea bazată pe reguli include transformările convenționale de eliminare a coloanelor redundante, eliminarea proiecțiilor, propagarea și derivarea predicatelor, eliminarea clauzelor GROUP BY redundante sau a join-urilor inutile [32]. Optimizarea bazată pe cost se face similar cu cea din CRDB, cu statistici pe tabel, în plus pe lângă statistici legale de numărul de rânduri, numărul de rânduri distince, numărul de valori null se folosește și un top de valori frecvente, aceste valori sunt recalculat prin funcția ANALYZE sau periodic [38]. Cel mai distinctiv mecanism de optimizare din TiDB este însă propagarea calculului către nivelul de stocare prin intermediul interfeței coprocessor: selecțiile, agregările parțiale, TopN, Limit și fragmentele de proiecție sunt evaluate în paralel în cadrul fiecărei Regiuni TiKV, iar doar rezultatele intermediare reduse sunt returnate nivelului de procesare pentru asamblarea finală [32].

Strategii de join Operațiile de join în topologia studiată (exclusiv pe rânduri) sunt efectuate în cadrul PD, deoarece coprocesoarele TiKV nu sunt capabile să efectueze joinuri între regiuni [39, 32]. Optimizatorul are de ales între 3 algoritmi. Joinuri hash, prin care planificatorul materializează partea de construcție într-o tabelă hash și o interoghează cu cealaltă parte, execuția lor este multi-thread. Dacă numărul estimat de rânduri care trebuie joinuite este redus, este preferabilă utilizarea metodei de join prin index [39]. Joinul prin interclasare (merge join) este un tip special de join care se aplică atunci când ambele părți sunt citite în ordine sortată. Această metodă nu folosește la fel de multă memorie ca un hash join dar nu poate fi executată în paralel.

Această concentrare a procesării joncțiunilor într-un nivel centralizat reprezintă un blocaj structural pentru interogările analitice care produc rezultate intermediare de mari dimensiuni și motivează decizia arhitecturală din implementările HTAP de a introduce un motor columnar. Această cale este exclusă în mod deliberat din comparația de față și este documentată ca atare în capitolul de metodologie.

3.3 Citus (Extensie PostgreSQL)

Arhitectura coordinator-worker, distributed vs reference tables Citus funcționează ca extensie PostgreSQL care interceptează planificarea cererilor prin hook-uri expuse de server. Fiecare nod din cluster (coordinator și workers) este un server PostgreSQL complet. La parsarea unei cereri, planificatorul Citus determină dacă sunt referențiate tabele Citus; în caz afirmativ,

produce un arbore de plan cu rădăcină CustomScan care encapsulează strategia de execuție distribuită [33]. Metadatele clusterului sunt stocate în cataloagele sistem prefixate cu `pg_dist_`, iar coordonatorul servește ca punct de intrare pentru conexiunile client.

Routarea și executarea interogărilor Planificatorul Citus clasifică cererile în categorii și selectează strategii corespunzătoare [33, 40]. Calea cea mai eficientă este cererea router: atunci când o cerere conține predicat de egalitate pe coloana de distribuție, planificatorul rescrie identificatorii și expediază enunțul către singurul worker cu fragmentele relevante, fără execuție pe coordonator.

Partiționare Coloana de distribuție este cea mai importantă decizie de design [41]: trebuie să aibă cardinalitate mare, distribuție uniformă (evitând asimetria fragmentelor), să apară frecvent în GROUP BY și join-uri. Pentru aplicații multi-tenant, un identificator de locatar pe fiecare tabel produce join-uri co-locate. Pentru analytics star-schema, tabelul de fapte și dimensiuni mari trebuie să partajeze un identificator comun, iar dimensiuni mici se convertesc la tabele de referință [41, 40].

3.4 Comparație arhitecturală sintetică

Cele trei sisteme se diferențiază pe două axe principale. Primul este cuplajul dintre nivelul SQL și motorul de stocare: strâns în CockroachDB (binar unic) [31]; slab în TiDB (noduri SQL comunică cu TiKV prin gRPC) [32]; bazat pe extensii în Citus (PostgreSQL nemodificat cu hook-uri) [33]. Al doilea este granularitatea gestionării replicării: CockroachDB și TiDB rulează grupuri Raft independente per fragment cu consensus per-shard, iar Citus (open-source) se bazează pe replicarea streaming PostgreSQL fără consensus per-shard.

Capitolul 4

Metodologie

Acest capitol descrie sistemul folosit pentru a evalua performanța interogărilor analitice ale CockroachDB, TiDB și Citus pe clustere de dimensiuni diferite. Fiecare decizie de design este justificată în termenii contribuției sale la validitatea comparației, de la alegerea workload-ului și setului de date, decizii luate legate de infrastructură și tipologia cluster-ului, până la pipeline-ul de încărcare a datelor, framework-ul de benchmarking (analiză comparativă) și protocolul de măsurare.

4.1 Utilizarea setului de date TPC-H și caracterizarea workload-ului

Pentru acest studiu a fost selectat benchmark-ul Transaction Processing Performance Council Decision Support (TPC-H) ca workload analitic (de tip OLAP). TPC-H modelează un scenariu de date pe o magazie în lanțul de aprovizionare, acesta cuprinde 22 de query-uri SQL (interogări) de citire aplicate asupra 8 tabele. Benchmark-ul a fost proiectat pentru a stresa sistemele de suport decizional, folosinduse de scanări complete sau parțiale ale tabelelor, join-uri multi-tabel peste seturi de rezultate intermediare mari, agregări și subquery-uri corelate.

Selecția unui benchmark analitic pentru a evalua sisteme al căror țință de proiectare primară este un workload tranzacțional se justifică prin faptul că CockroachDB, TiDB și Citus se poziționează ca fiind capabile să deservească workload-uri mixte sau hibride. În documentația acestor sisteme se discută suportul pentru interogări analitice. Evaluarea acestor sisteme sub un set de interogări pur analitice introduce astfel o discriminare din punct de vedere arhitectural pentru motoarele distribuite orientate pe rânduri. Utilizarea unui benchmark tranzacțional ar măsura în schimb coordonarea blocajelor și latența commit-ului, domeniile în care aceste sisteme sunt optimizate, dar ar produce rezultate mai puțin revelatoare ale compromisurilor arhitecturale investigate în contextul interogărilor grele din punct de vedere analitic.

4.1.1 Schema relațională

Schema TPC-H este formată din 8 tabele care exprimă o parte dintr-un lanț de aprovizionare și anume o magazie: PART, SUPPLIER, PARTSUPP, CUSTOMER, ORDERS, LINEITEM, NATION, REGION și LINEITEM (tabelul principal), la un SF30 (scale-factor - factor de scară), acest tabel are aproximativ 180 milioane de rânduri (în jur de 22GB de date), marcând acest tabel ca cel principal în majoritatea interogărilor care folosesc scan și join-uri. Restul tabelelor, ORDERS și PARTSUPP sunt tabele de mărime medie, pe când NATION și REGION sunt tabele de referințe mici de 25 respectiv 5 rânduri.

4.1.2 Scale Factor

A fost utilizat un scale factor (factorul de scară) 30 (SF30), aproximativ 30GB de date ne-comprimate. Această decizie reflectă unele limitări introduse de infrastructura de testare, dacă foloseam un set mai mic de date, mai ales într-un sistem de 20 de noduri, exista riscul ca o interogare agregată mare să fie stocată într-un buffer (zonă de memorie temporară) și să fie servită din această memorie, mascând astfel interogările I/O și transferurile din rețea. Dacă foloseam un set mai mare de date exista riscul să nu încapă pe numărul mic de noduri testat.

4.2 Clasificarea interogărilor TPC-H

Am grupat cele 22 de interogări din TPC-H în patru categorii bazat pe execuția acestora. Această grupare este aplicată în această lucrare în analiza rezultatelor pentru a determina dacă tiparul observat în diferențe se corelează cu clasa complexității interogării sau reflectă un comportament al sistemului în întreg.

Grupa 1 - Interogări dominate de scanări (Q1, Q6, Q14, Q19). Aceste interogări sunt dominate de o singură secvență de scanare asupra tablei LINEITEM, care conține aproximativ 180 de milioane de rânduri. Q1 și Q6 efectuează o scanare urmată de o agregare scalară sau grupată. Q14 introduce un join pe tabela mai mică part. Q19 adaugă predicate complexe care combină mai multe condiții.

Grupa 2 - Interogări dominate de îmbinări (Q3, Q5, Q7, Q8, Q9, Q10). Aceste interogări execută joins pe mai multe tabele, variind de la trei la șase relații. Q5, Q8 și Q9 sunt cele mai complexe interogări, îmbinând mai multe tabele de dimensiuni multiple, inclusiv LINEITEM. Q7 include un join încrucișat pe tabela NATION. Performanța acestor interogări va depinde de strategia de join selectată de planificator și capacitatea sistemului de a redistribui date între noduri.

Grupa 3 - Interogări conduse de subinterogări (Q2, Q11, Q17, Q22). Aceste interogări se folosesc de subinterogări care calculează praguri pentru predicatele de filtrare interogărilor

externe.

Grupa 4 - Interogări în mai multe etape (Q4, Q12, Q13, Q15, Q16, Q18, Q20, Q21). Aceste interogări se execută ca etape multiple cu materializare intermediară și dependențe. Q4 și Q21 utilizează semi-joins și anti-joins. Q12 și Q18 înlănțuiesc scanări și joins. Q13 și Q15 efectuează operații de grupare consecutive. Q16 combină un anti-join cu o operație de distinct. Q20 exprimă condiții corelate rescrise în lanțuri de joins.

4.3 Infrastructură

4.3.1 Specificația hardware și definirea nodurilor

Infrastructura pentru cluster a fost provizionată pe furnizorul Hetzner Cloud și formată din două tipuri de noduri: un singur *controller* nod și un număr variabil de noduri pentru *baza de date*. **Nodul controller** a rulat pe o instanță CPX32 cu următoarele specificații 4 vCPU AMD, 8GB RAM, 160GB local NVMe SSD și a fost nodul principal care a deservit mai multe funcționalități în acest benchmark precum: rularea aplicației, generarea și stocarea datelor TPC-H până la distribuirea în baza de date, și infrastructura pentru distribuirea datelor în momentul încărcării. **Nodurile pentru baza de date** au rulat fiecare pe o instanță cpx52 cu următoarele specificații: 12 vCPU AMD, 24 GB RAM, 480GB local NVMe SSD. Toate nodurile pentru baza de date au fost provizionate cu același tip de server pe toate sistemele de baze de date și numărul de noduri testat. Aceste statistici hardware au fost confirmate manual prin verificarea lor pe server și prin statisticile furnizate în momentul colectării de la TiDB. Toate nodurile au rulat o imagine goală de Ubuntu 22 (forțat de către Citus care nu suporta la momentul testării Postgres18 și Ubuntu 24 prin versiunea Citus 4).

Instanțele pentru nodurile bazei de date a fost aleasă ca fiind cpx52 în urma experimentării inițiale pe noduri mai mici (cpx32) care au eșuat în a rula anumite interogări complexe rezultând o eroare de tip OOM (out of memory - procesul rămânea fără memorie RAM).

Setările pentru fiecare bază de date au fost listate în fișierele de inițializare, s-a încercat păstrarea lor cât mai aproape de setările implicite. În cazul TiDB a fost mărită memoria disponibilă pentru o interogare pentru a prevenii OOM-ul, iar pentru Citus este nevoie să setezi această memorie, nu există o setare implicită.

4.3.2 Arhitectura rețelei

Toate nodurile au fost interconectate printr-o rețea privată în cadrul Hetzner în spațiul adresei 10.0.0.0/16, cu un subnet dedicat pentru comunicarea internă a clusterului 10.0.1.0/24. Controller a avut tot timpul IP-ul fix 10.0.1.10, iar nodurile pentru bazele de date au primit ip-uri în ordine crescătoare de la 10.0.1.20. Nodurile pentru baza de date nu au avut o adresă

IPv4 publică, ele nefiind accesibile din rețeaua publică decât printr-un SSH jump prin controller. Această implementară mimează o implementare reală în producție al unui astfel de sistem.

4.3.3 Provizionare și automatizare

Crearea infrastructurii a fost automatizată prin intermediul Terraform, cu pluginul de Hetzner Cloud Provider oferit de ei. Un singur fișier `main.tf` a definit rețeaua privată, nodul controller și nodurile de baze de date printr-un parametru pentru numărul lor. Fiecare cluster a fost inițializat pentru benchmark și șters după acesta pentru a optimiza costurile și a prevenii interferențele între sistemele de baze de date, astfel fiecare rulare a fost făcută pe o instalare proaspătă.

Automatizarea pentru configurarea serverelor a fost formată dintr-un număr de scripturi shell, executate atât local (primul script) cât și prin SSH pe nodul controller. Primul script și anume `upload-benchmark.sh` a fost rulat de pe laptopul personal, pentru a transfera toate fișierele de care este nevoie pe nodul controller și anume: aplicația de benchmark compilată (fișierul `.JAR`), interogările pentru TPC-H (specifice pentru fiecare motor de baze de date), schema bazei de date, celălalte script-uri pentru setupul nodurilor, un fișier de environment (`.env`) și un script pentru generarea datelor TPC-H. În acest script se mai execută și anumite comenzi pentru a instala dependențele necesare pe nodul controller precum: Docker, NGiNX, Java și GCC - pentru compilarea utilității TPC-H oferită de TPC. La finalul acestui script se distribuie cheia SSH publică a nodului controller pe fiecare nod de baze de date pentru acces ulterior.

După acest proces, am folosit următorul script și anume `setup-all-nodes.sh`, acest script iterează prin fiecare nod și face preparări la nivel de sistem de operare și efectuează o restartare. În final se așteaptă ca fiecare nod să repornească și se dă o restartare finală pentru nodul controller.

Pentru a rula benchmarkul se folosește un script numit `run-benchmark.sh` care invocă alte 4 scripturi la rândul său, fiecare individuale pentru fiecare sistem de baze de date și anume: dezinstalarea sistemului, instalarea lui, încărcarea datelor, colectarea metadatelor (setărilor din baza de date și analizele interogărilor), rularea aplicației și apoi dezinstalarea sistemului din nou. Dezinstalarea sistemului este o parte ce nu a fost utilizată, deoarece noi refăceam tot cluster-ul la orice modificare a numărului de noduri sau a sistemului testat.

4.3.4 Instalarea bazei de date

Cum am spus mai sus, fiecare sistem avea proprii pași pentru instalare, prin propriile scripturi operate via SSH din nodul controller.

Pentru **CockroachDB** binarul de instalare a fost descărcat pe nodul controller, apoi distribuit prin `scp` pe fiecare nod de bază de date. După ce baza de date a fost instalată pe fiecare nod a fost realizată o înregistrare în `systemd` pentru procesul Cockroach. Inițializarea clusterului a fost realizată pe nodul primar (10.0.1.20) prin comanda `start`, după ce toate sistemele au fost accesibile și disponibile pentru conectare.

TiDB a fost instalat prin sistemul TiUP oferit de PingCAP, fiind recomandarea oficială pentru pornirea acestui sistem. Un fișier de tip YAML a fost generat pe nodul controller în funcție de numărul de noduri pentru a stabili componentele ce vor forma clusterul (TiKV, PD, TiDB). Acest fișier a fost folosit de către tiup prin comenzile `cluster deploy` și `cluster start` pentru a da drumul sistemului, făcând acest sistem cel mai ușor de instalat.

Citus a fost instalat pe fiecare nod, folosind script-ul oficial oferit de cei de la Citus folosindu-se pachetul `postgresql-17-citus-13.0` prin `apt`. Fișierele necesare și anume `postgresql.conf` și `pg_hba.conf` au fost modificate prin intermediul SSH herodoc pentru a configura inițializarea în rețea pentru subrețeaua `10.0.0.0/16`. Nodul coordonator (`10.0.1.20`) a fost configurat apoi prin comenzile `citus_set_coordinator_host` și `citus_add_node`.

4.4 Încărcarea datelor

Fișierele pentru datele TPC-H au fost generate pe nodul controller, folosind kit-ul oferit de TPC dbgen la SF30. Pentru fiecare tabel s-a produs un fișier `.tbl`. Pentru fiecare sistem s-a implementat o metodă diferită de încărcare a datelor, doar fișierul pentru tabelul `LINEITEM` avea în jur de 22GB.

Pentru sistemele CRDB și TiDB a fost considerată utilizarea comenzilor de încărcare date locale (prin distribuirea fișierelor prin SCP către nodul principal și utilizarea `LOAD DATA LOCAL INFILE` dar aceasta s-a dovedit neutilizabilă pentru volumul de date folosit.

4.4.1 CockroachDB: Import HTTP prin NGiNX

Pentru CockroachDB am folosit comanda `IMPORT INTO` care încarcă date din fișiere CSV prin URL-uri accesibile prin protocolul HTTP.

O instanță de NGiNX a fost configurata pe nodul controller pentru a servi directorul cu fișierele TPC-H prin HTTP pe portul 8080. Fiecare execuție `IMPORT INTO` a fost făcută printr-un URL de forma:

```
http://10.0.0.1:8080/tpch/<table>.tbl
```

Pentru tabelele a căror fișiere `.tbl` au depășit dimensiunea de 500MB, fișierul a fost împărțit în mai multe bucăți folosind comanda din Unix `split` în bucăți de câte 2GB fiecare. Fișierele rezultate au fost apoi distribuite către CRDB prin comanda `IMPORT INTO` printr-o listă de URL-uri despărțite de o virgulă, astfel permițând sistemului să importe în paralel conținutul, distribuind I/O pe cluster. Acest lucru a fost relevant pentru fișierele mari precum `LINEITEM`, `ORDERS` și `PARTSUPP`.

4.4.2 TiDB: Compatibil S3 prin MinIO

Documentația TiDB sugerează distribuirea funcției `IMPORT INTO` prin citirea datelor de pe o soluție compatibilă cu protocolul S3. TiDB distribuie aceste date prima dată către nodul de calcul (TiDB) și apoi distribuie acestora către sistemul de stocare (TiKV).

O instanță de MinIO (serviciu compatibil S3) a fost pornită prin docker pe nodul controller furnizând astfel adrese compatibile cu protocolul S3 de către nodul controller. Un sistem similar a fost utilizat cu cel din CRDB, fiecare comandă de `IMPORT INTO` a fost trimisă cu URL-uri compatibile s3:// și distribuită către încărcare de către baza de date. Importurile au fost realizate prin job-uri async, folosind un poll-ing sistem pentru a monitoriza statusul importurilor o dată la 5 secunde, încărcându-se câte un tabel pe rând.

4.4.3 Citus: Transfer direct de fișier

Citus, fiind o extensie peste PostgreSQL, moștenește comanda `COPY`, aceasta a fost folosită pentru import. Față de celelalte sisteme, această comandă are nevoie ca fișierele să existe pe primul nod, astfel toate fișierele `.tbl` au fost transferate prin `scp` către nodul primar. După import Citus crează automat shard-urile pentru distribuția coloanelor.

4.5 Frameworkul de benchmarking

4.5.1 Implementare

Infrastructura de benchmarking a fost implementată în Java, utilizând Java Database Connectivity (JDBC) API pentru comunicările cu toate sistemele de baze de date. JDBC furnizează un API abstractizat pentru conexiunile cu toate sistemele, fără a fi necesare alte modificări, CRDB și Citus se folosesc de protocolul pentru PostgreSQL, folosind driver-ul JDBC pentru PSQL. TiDB se folosește de protocolul MySQL astfel folosindu-se driver-ul MySQL.

Rularea acestei aplicații este configurabilă prin fișierul `.env` și argumente de rulare. Există o variabilă `ACTIVE_DB` care stabilește ce conexiune se folosește pentru baza de date `COCKROACH`, `TIDB`, `CITUS` și alege ce parametri să folosească pentru datele de conectare. De exemplu pentru `COCKROACH` se folosesc următorii parametri: `COCKROACH_HOST`, `COCKROACH_PORT`, `COCKROACH_DB`, `COCKROACH_USER` și `COCKROACH_PASSWORD`. Argumentul `--nodes` a fost folosit pentru a salva informații despre numărul de noduri în fișierul de rezultate, el neafectând altfel funcționalitatea aplicației. Ultimele variabile folosite și anume `QUERIES_PATH` și `RESULTS_PATH` controlează locația în care sunt stocate fișierele SQL cu interogări și locul în care rezultatele în format CSV sunt salvate.

URL-ul pentru conexiunea JDB este construit per fiecare tip de bază de date în cadrul implementării clasei conectoare.

CRDB utilizează:

```
jdbc:postgresql://host:port/db?sslmode=disable
```

Citus utilizează:

```
jdbc:postgresql://host:port/db
```

TiDB utilizează:

```
jdbc:mysql://host:port/db?useServerPrepStmts=true
```

Interfața `DatabaseConnector` este una de tip sealed, care permite exact 3 implementări și anume cele 3 descrise mai sus, selectate în momentul executării printr-un switch dintr-un enum `DatabaseType` creat în momentul inițializării la parsarea fișierului de `.env`.

Programul a fost rulat exclusiv pe nodul controller după ce a fost compilat într-un executabil JAR. Nodurile de bază de date nu au avut de loc legătură cu orchestrarea acestui program. Outputul generat de către acest program a fost salvat pentru fiecare rulare într-un fișier `run_<db>_<N>_nodes.log` pe controller. Fișierele de log, metadata și rezultate au fost apoi descărcate via scp.

4.5.2 Managementul conexiunii și izolarea

Pentru conexiune nu a fost utilizat un connection pool, pentru fiecare rulare, inițială sau de măsurare, driver-ul JDBC deschide o conexiune nouă, descarcă rezultatul interogării și închide conexiunea. Această decizie este intenționată pentru a nu contamina măsurările de latență pentru răspunsul unei interogări. Costul conexiunii este consistent pentru fiecare măsurare pentru fiecare sistem, astfel nu introduce o eroare în măsurătorile obținute.

Acesta este codul principal care execută rulările de măsurare, se poate observa mecanismul de izolare a conexiunilor utilizând construirea unei resurse prin intermediul constructorului `try-with`:

Cod 4.1: Ciclu de viață al unei conexiuni

```
var durations = new ArrayList<Long>();
for (int i = 0; i < appConfig.measurementRuns(); i++) {
    try (var conn = connector.connect()) {
        connector.clearCache(conn);
        long startNanos = System.nanoTime();
        executeQuery(conn, query.sql());
        long durationNanos = System.nanoTime() - startNanos;
        durations.add(durationNanos);
    } catch (SQLException e) { e.getMessage(); }
}
```

Putem observa o chemare a unei funcții `clearCache`, această funcție este definită în interfața `DatabaseConnector` și este invocată în fiecare rulare. În implementarea curentă această funcție este un noop, deoarece nici unul dintre sisteme nu expune un mecanism consistent pentru a

curăța cache-ul sau buffer poolul. Principalul mecanism utilizat pentru a ignora modificările rezultate din cache este reînnoirea conexiunii. Pentru conexiune nu este implementat nici un fel de timeout, astfel orice execuție așteaptă un răspuns oricât este nevoie. Singurele terminări ale execuției unei interogări pot provenii de la sistemul de baze de date, care au fost lăsate nemodificate și neconfigurate (configurarea default) pentru fiecare sistem.

4.5.3 Colectarea rezultatelor din program și exportul

Fiecare rulare salvează o măsurătoare definită prin clasa record `BenchmarkResult` care salvează informații despre sistemul de bază de date, un identificator unic pentru interogare, numele datasetului folosit, numărul de noduri și durata de timp individuală pentru fiecare rulare salvată în nanosecunde. Aceasta este ulterior convertită în milisecunde în momentul exportării. Aceste rezultate sunt salvate într-un fișier CSV de către clasa `CsvExporter` cu următorul format:

```
database,query_id,query_name,query_dataset,node_count,duration_ms
```

Toată procesarea de date ulterioară pentru compilarea rezultatelor și graficelor precum media, mediana, deviația standard, minimum și maximum per interogare per configurație au fost calculate ulterior într-un Jupyter Notebook folosind Python și pandas.

4.6 Metodologia de măsurare

4.6.1 Warmup runs (rulări inițiale)

Înainte de a rula orice interogare măsurabilă care să salveze un rezultat, se execută un număr de warmup runs (3 în cazul nostru), aceste rulări folosesc același model descris mai sus pentru rulările de măsurare, o conexiune nouă este deschisă, interogarea se execută complet, inclusiv prin parcurgerea rezultatului iar la urmă conexiunea este închisă. Singura diferență este că acest timp nu este înregistrat niciunde. Aceste rulări se folosesc pentru a lăsa baza de date să pregătească pentru executările măsurabile prin stabilirea planurilor optime și salvarea acestora și stabilirea diferitelor conexiune necesare, astfel prevenim creșterea nesemnificativă a primei rulări de măsurare pentru fiecare interogare. Aceste rulări sunt executate de 3 ori pentru fiecare query, pe fiecare configurație de cluster, înainte de rularea măsurată.

4.6.2 Rulările de măsurare

După rularea inițială pentru warmup, se execută câte 10 rulări măsurate pentru fiecare interogare în fiecare configurație pentru fiecare sistem, în total rezultând $22 \times 3 \times 4 \times 10 = 2640$ rulări cronometrate individuale. Numărul de 10 rulări este considerat suficient pentru a obține o statistică mediană pentru comparație și pentru identificarea valorilor aberante din distribuție,

păstrând totodată o fezabilitate din punct de vedere computațional, având în vedere durata de mai multe ore a sesiunilor individuale de rulare a benchmarkului.

Statistica primară pentru raportare o reprezintă timpul median de execuție, nu media aritmetică. Mediana este preferată în detrimentul mediei aritmetice deoarece distribuțiile latenței de execuție a interogărilor în sistemele distribuite sunt asimetrice, blocajele de rețea, alegerea distribuirii pot genera rulări aberante care măresc media aritmetică fără a reprezenta comportamentul tipic al sistemului. Însă media aritmetică, abaterea standard, minimumul și maximumul sunt și ele calculate pentru a caracteriza rezultatele în analizele comparative.

4.7 Fairness și comparabilitate

Au fost luate mai multe decizii de design care au fost îndreptate spre a crea un mediu în care diferențele observate reflectă diferențele arhitecturale între sisteme.

Toate sistemele au fost evaluate folosind standardul TPC-H, fără a fi alterate interogările. Nu a fost efectuată nici o modificare deși o parte dintre furnizorii de servicii cloud propun rescrierea unor interogări pentru adaptarea lor mai bine în sistemele distribuite, pentru a lucra în jurul planificatorului.

Statisticile pentru tabele au fost generate o dată, pentru fiecare cluster, la sfârșitul încărcării datelor prin comanda ANALYZE înainte de rularea interogărilor.

Benchmarkul a fost rulat cu o singură conexiune, fără concurență. Această decizie a fost luată intenționat, o unică conexiune către baza de date izolează studiul variabilelor de distribuite și elimină influența blocărilor într-un workload concurent. O analiză asupra concurenței reprezintă un alt studiu individual în sine, folosindu-se de alte metrice și este identificat ca o direcție viitoare pentru cercetare.

Interogările ce nu au putut fi executate pe anumite sisteme, fie din cauza limbajului SQL (anumite interogări au fost adaptate pentru rularea în MySQL pentru TiDB) sau din cauza arhitecturii, în special (Citus) apar în toate statisticile cu 0 timp de rulare.

Capitolul 5

Rezultate obținute

Acest capitol prezintă rezultatele obținute pe cele trei sisteme analizate, CockroachDB, TiDB și Citus, în patru configurații descrise mai devreme de cluster cu 3, 5, 10 și 20 de noduri.

Cele 22 de interogări au fost împărțite în patru grupe, definite anterior, pe baza profilului dominant al interogării. Fiecare secțiune de grupă este structurată în jurul a patru grafice generate din datele prelucrate cu `pandas` și `matplotlib`. Anumite celule din grafice apar marcate „N/A”, aceste lipsuri nu reprezintă erori de măsurare, ci constatări arhitecturale documentate.

5.1 Grupa 1 - Interogări dominate de scanări

Prima grupă, definită anterior este formată din interogările Q1, Q6, Q14 și Q19, ale căror planuri de implementare în toate sistemele de baze de date sunt similare și sunt dominate de o scanare integrală sau filtrată a celei mai mari tabelă din benchmark, `lineitem`. Q1 calculează o agregare pe două coloane de grupare cu un filtru pe `l_shipdate`. Q6 selectează toată tabela `lineitem` și o reduce la un singur scalar pe baza mai multor predicate, Q14 efectuează un join selectiv cu part dar conform planurilor de execuție, timpul principal este petrecut în executarea scanării tabelului `lineitem` filtrat pe dată. Q19 efectuează o îmbinare similară cu cea din Q14 dar aplică predicate de filtrare complexe. În toate cele 4 interogări, costul dominant este parcurgerea celor aproximativ 180 de milioane de rânduri ale tabelului `lineitem` și aplicarea unor filtre relativ simple. Acest grup indică informații despre eficiența motorului de scanare și strategia de distribuire a tabelului voluminos pe mai multe noduri.

Grupa 1 - Interogări dominate de scanări: Latență mediană vs Număr de noduri

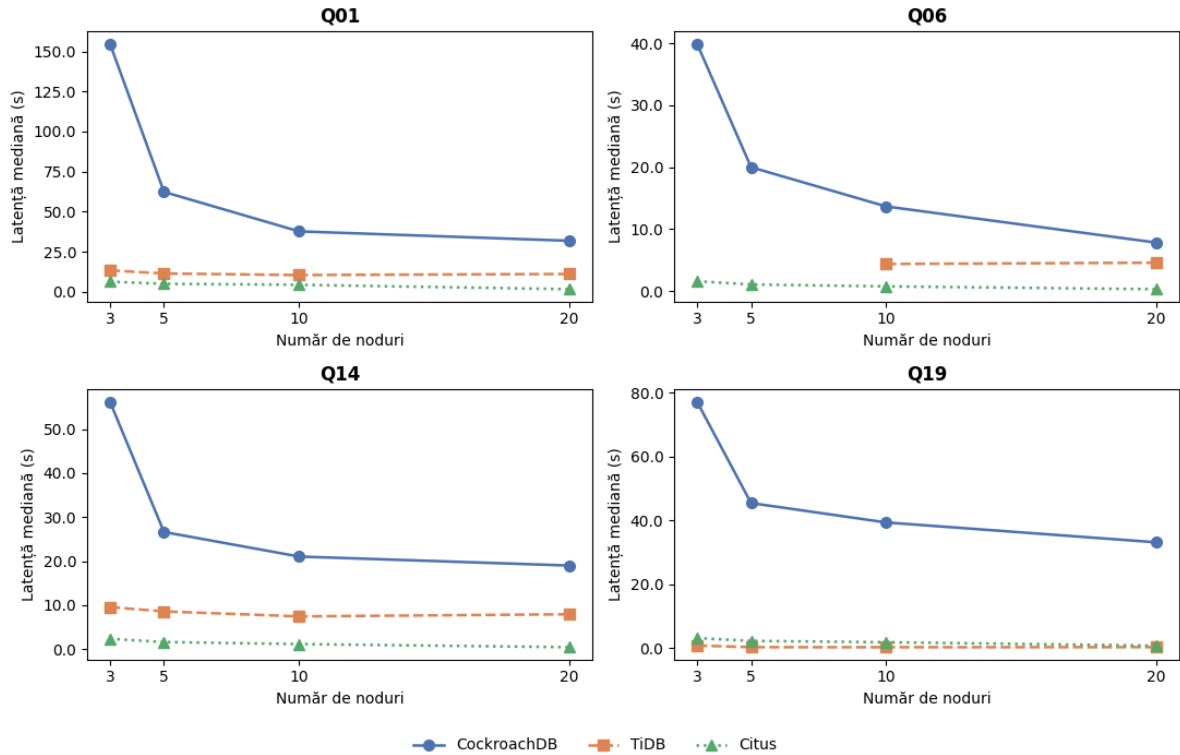


Figura 5.1: Grupa 1 - Interogări dominate de scanări: latența mediană în funcție de numărul de noduri pentru fiecare interogare.

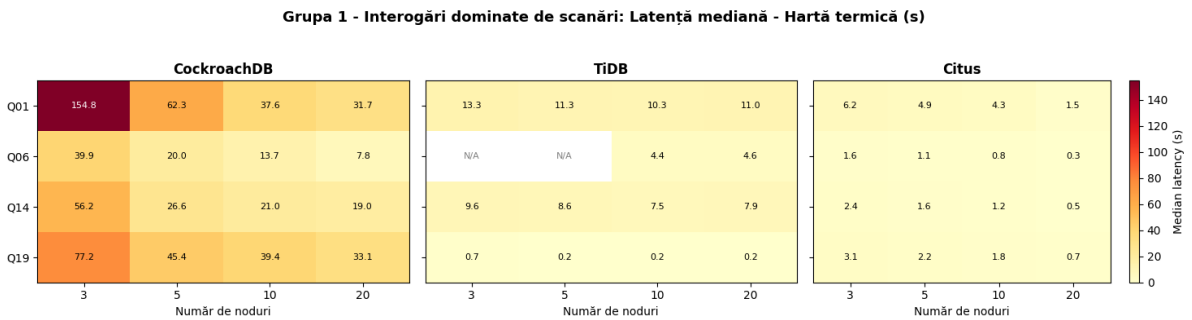


Figura 5.2: Grupa 1 - Interogări dominate de scanări: harta termică a latenței mediane (secunde) pe matricea sistem × număr de noduri.

Figura 5.1 și harta termică din figura 5.2 arată o ierarhie consistentă a latenței mediane. Citus este cel mai rapid sistem la fiecare combinație măsurată. TiDB se situează la mijloc, iar CockroachDB are cea mai mare latență cu o marjă semnificativă. Se poate observa o diferență majoră la configurația de 3 noduri, Q1 rulează în $6,2 \approx$ secunde pe Citus și în $13,3 \approx$ secunde pe TiDB, dar necesită $154,8 \approx$ secunde pe CRDB, un factor de aproximativ $25\times$ în defavoarea acestuia. Aceste tipare se observă și pe Q14 ($2,4$ vs $9,6$ vs $56,2 \approx$ secunde) și pe Q19 ($0,7$ vs

3,1 vs 77,2 $\approx$ secunde). Este de notat că la Q19 TiDB se apropie de performanța Citus. La configurația cea mai puternică, cea de 20 de noduri distanța dintre CRDB și restul sistemelor se reduce dar nu se elimină, ierarhia rămâne aceeași pe toate interogările din acest grup.

Grupa 1 - Interogări dominate de scanări: Accelerație relativă (linie de bază: 3 noduri)

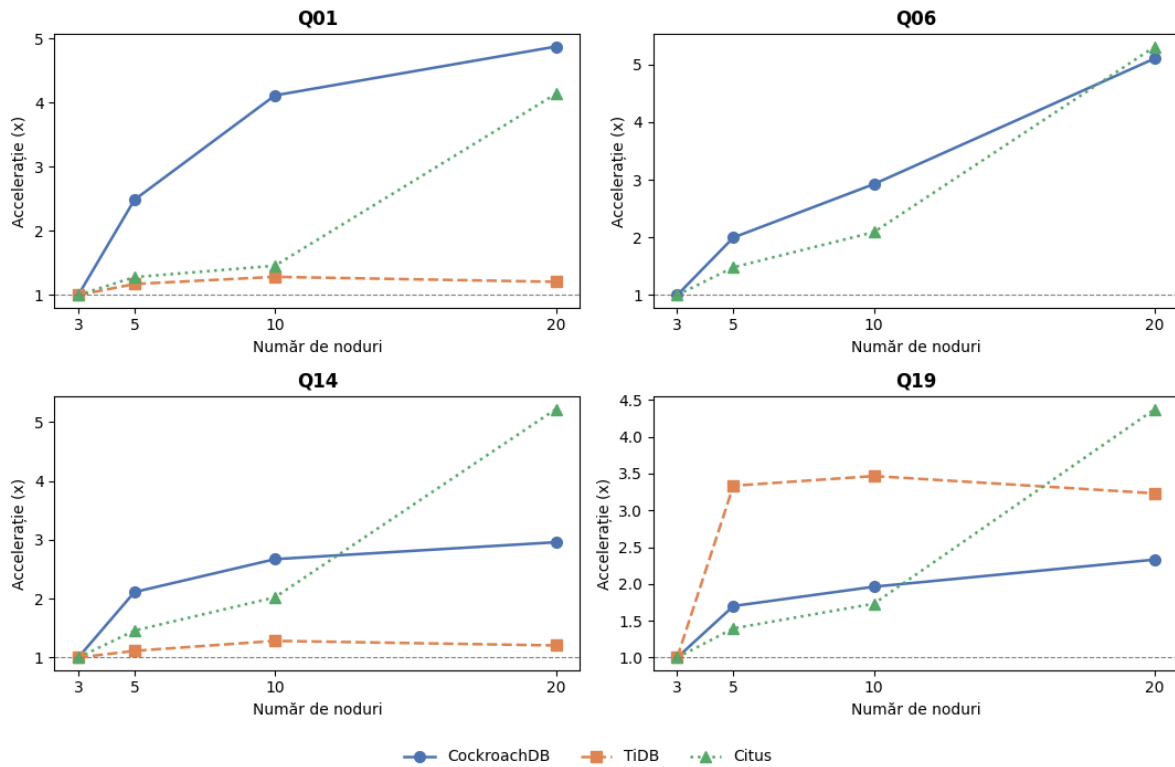


Figura 5.3: Grupa 1 - Interogări dominate de scanări: accelerația relativă la linia de bază cu 3 noduri.

Analiza accelerației relative la configurația de 3 noduri, prezentată în figura 5.3, completează imaginea. CockroachDB prezintă cea mai pronunțată accelerație în această grupă, atingând chiar un factor de aproximativ 4,9 $\times$ pe Q1 și 5.1 $\times$ pe Q6. Citus prezintă un comportament similar. Această observație este interesantă, aproape contradictorie cu analiza latențelor dar este revenită din planurile de execuție. CockroachDB scalează bine pentru că pornește de la o bază lentă, cu mult spațiu disponibil pentru paralelizare. În contrast, TiDB prezintă o accelerație destul de slabă, singura excepție fiind reprezentată de Q19, unde TiDB atinge 3,4 $\times$ accelerație la 5 noduri, dar apoi se stabilizează, indicând o îmbunătățire provenită din nivelul de paralelizare.

Grupa 1 - Interogări dominate de scanări: Distribuția execuțiilor - 10 noduri

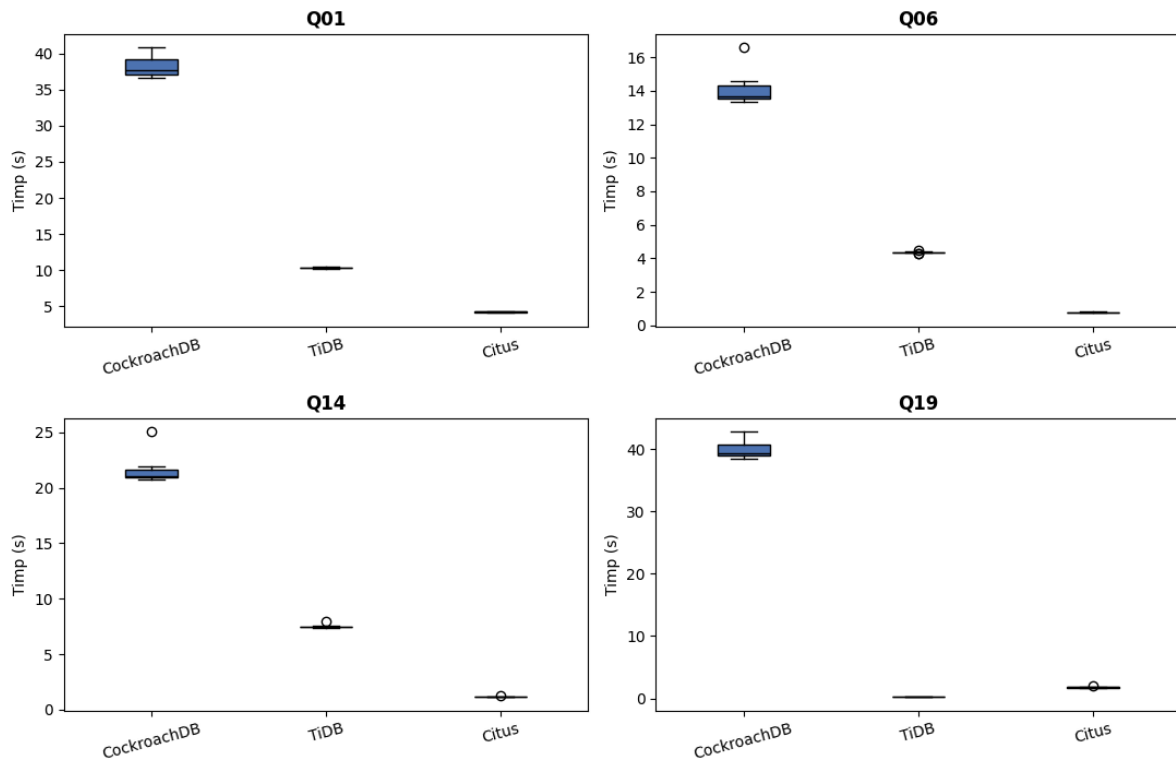


Figura 5.4: Grupa 1 - Interogări dominate de scanări: distribuția celor zece rulări temporizate la configurația cu 10 noduri.

Distribuția rulărilor la 10 noduri, prezentată în figura 5.4, indică o variabilitate scăzută pentru toate cele trei sisteme pe această grupă. Diagramele de tip cutie sunt foarte strâmte pe TiDB și pe Citus, sugerând că aceste sisteme produc durate de execuție extrem de reproductibile pentru interogările dominate de scanări. CRDB prezintă cutii ușor mai late și cu valori aberante ocazional, dar variația rămâne sub 10 % din mediana.

Există două abateri notabile de la tiparul general. Prima este absența datelor pentru Q6 pe TiDB la configurațiile cu 3 și 5 noduri, marcată „N/A” în figura 5.2. Această absență nu reflectă un comportament arhitectural al sistemului, ci un eșec de colectare a planului la acele puncte specifice de măsurare, datorat unei erori OOM. A doua abatere este Q19, unde TiDB devansează atât CockroachDB cât și Citus în absolut, cu latențe sub o secundă la toate configurațiile de peste 3 noduri.

5.2 Grupa 2 - Interogări dominate de îmbinări

Cea de a doua grupă este formată din interogările Q3, Q5, Q7, Q8, Q9 și Q10, care combină tabelele `lineitem` și `orders` cu mai multe tabele de dimensiuni diferite prin `join`-uri multiple. Specific acestei categorii este costul dominant provenit din execuția unui graf de îmbinări, care

implică între 3 și 6 tabele simultan. Această grupă prezintă comportamentul fiecărui sistem la nivel arhitectural prin strategia de partiționare și capacitatea planificatorului de a alege un algoritm de join optim pentru a exploata co-localizarea datelor.

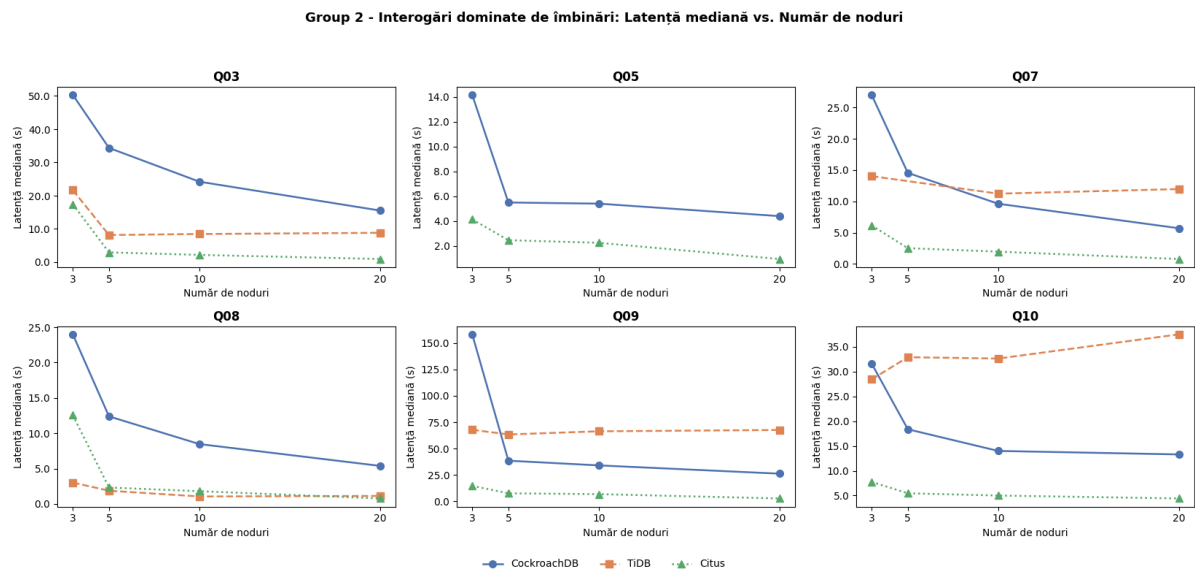


Figura 5.5: Grupa 2 - Interogări dominate de îmbinări: latența mediană în funcție de numărul de noduri.

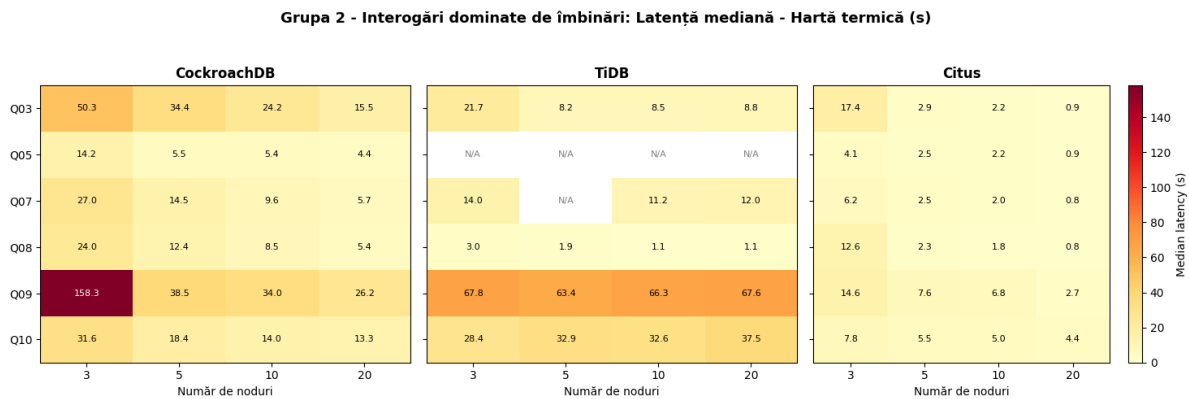


Figura 5.6: Grupa 2 - Interogări dominate de îmbinări: harta termică a latenței mediane (secunde).

Figura 5.5 și harta termică din figura 5.6 arată o ierarhie de performanță care este total diferită de cea din grupa precedentă. Citus rămâne în continuare sistemul cu cea mai mică latență, cu un avantaj mai ales pe Q3 (17,4 vs 21,7 vs 50,3 $\approx$ secunde) respectiv 0,9 vs 8,8 vs 15,5 $\approx$ secunde în configurația de 20 de noduri. În schimb, raportul dintre TiDB și CockroachDB se inversează față de prima grupă, pentru cele mai multe interogări din această grupă CRDB are latențe mai mici decât TiDB la configurațiile cu 5, 10 și 20 de noduri. Observăm pe Q9 cum CockroachDB scade de la 158,3 $\approx$ secunde în configurația de 3 noduri la 26,2 $\approx$ secunde în

configurația de 20 de noduri, în timp ce TiDB stagnează în jurul valorii de $66 \approx$ secunde în toate configurațiile. Tipare similare observă și la Q7 și Q10. Această observație reflectă diferența fundamentală dintre sarciniile dominate de scanare (unde pushdown-ul de agregare către stratul de stocare - TiKV este factorul decisiv) și sarciniile dominate de join-uri (unde algoritmul de îmbinare la stratul de execuție este factorul decisiv).

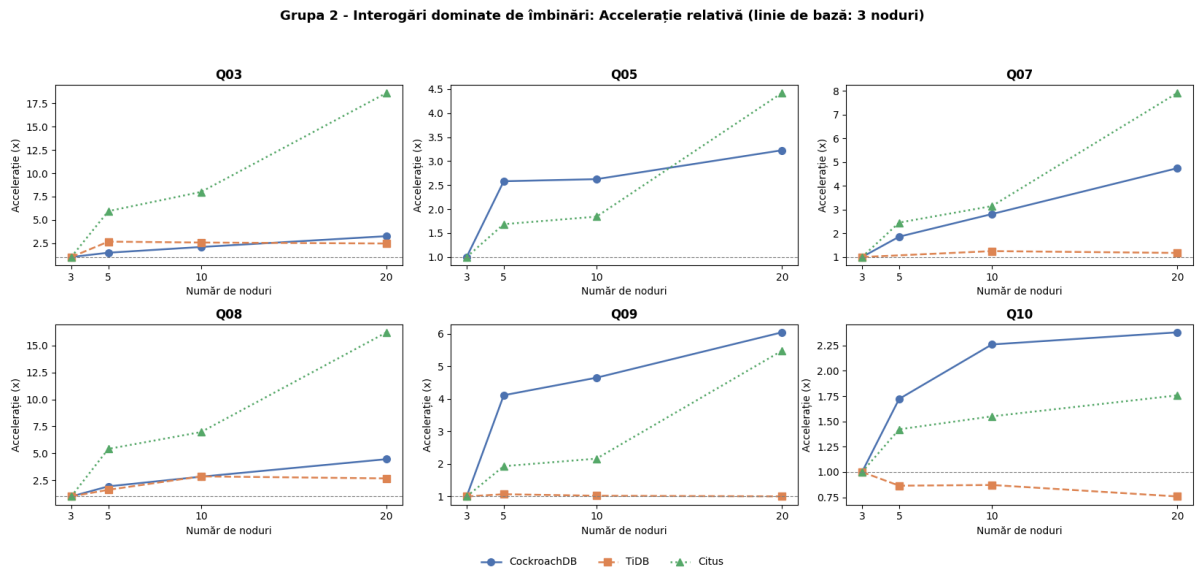


Figura 5.7: Grupa 2 - Interogări dominate de îmbinări: accelerația relativă la linia de bază cu 3 noduri.

Analiza accelerației din figura 5.7 amplifică această diferență. Citus prezintă factorii de accelerație cei mai mari din întreg studiul pe interogările Q3 (aproximativ $19\times$ la 20 de noduri) și Q8 (aproximativ $16\times$), iar CockroachDB înregistrează accelerații robuste pe majoritatea interogărilor din grupă, atingând $6\times$ pe Q9 și aproximativ $5\times$ pe Q7. Se poate observa comportamentul TiDB pe această grupă, unde pe Q8, factorul de accelerație rămâne practic egal cu 1 pe toate configurațiile, iar pe Q10 apare o regresie a performanței pe măsură ce numărul de noduri crește. Această observație este o consecință directă a algoritmului de join ales de sistemul TiDB.

Grupa 2 - Interogări dominate de îmbinări: Distribuția execuțiilor - 10 noduri

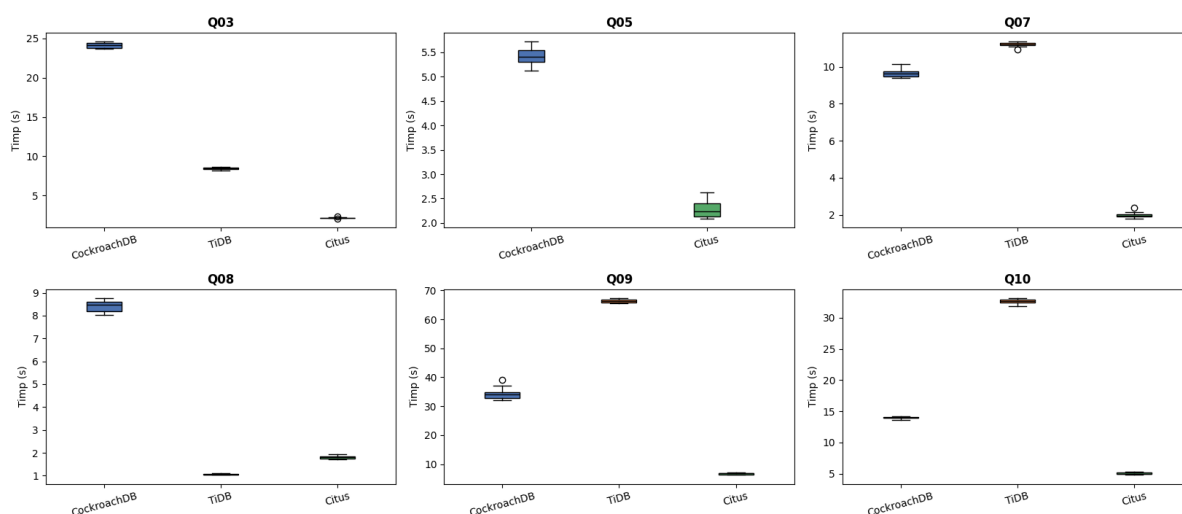


Figura 5.8: Grupa 2 - Interogări dominate de îmbinări: distribuția celor zece rulări temporizate la configurația cu 10 noduri.

Distribuția rulărilor la 10 noduri, prezentată în figura 5.8, confirmă că diferențele de latență observate sunt structurale și nu efecte ale variabilității. Cutiile sunt strâmte pentru toate cele trei sisteme pe majoritatea interogărilor, cu amplitudini relative sub 5%.

5.3 Grupa 3 - Interogări conduse de subinterogări

A treia grupă reunește interogările Q2, Q11, Q17 și Q22, fiecare dintre aceste interogări fiind caracterizate printr-o subinterogare corelată sau printr-o agregare cu un subansamblu derivat din tabele exterioare (de bază). Q2 se folosește de o subinterogare corelată pe `partsupp` pentru a căuta furnizorul cu costul minim pentru fiecare combinație între piesă și regiune. Q11 folosește o subinterogare scalară tot pe `partsupp` pentru a calcula piesele a căror valoare totală în stoc depășește un anumit prag. Q17 calculează venitul mediu pe piese cu cantități sub un prag derivat din media aceleiași tabele `lineitem`. Q22 segmentează clienții fără comenzi după zona telefonică, după filtrarea pe baza unei medii globale a soldului. În toate aceste cazuri, planul logic include cel puțin o subinterogare al cărui rezultat trebuie decorelat sau materializat înainte de a putea fi folosit în îmbinarea principală, iar performanța depinde critic de strategia aleasă de optimizator pentru această materializare.

Grupa 3 - Interogări conduse de subinterogări: Latență mediană vs. Număr de noduri

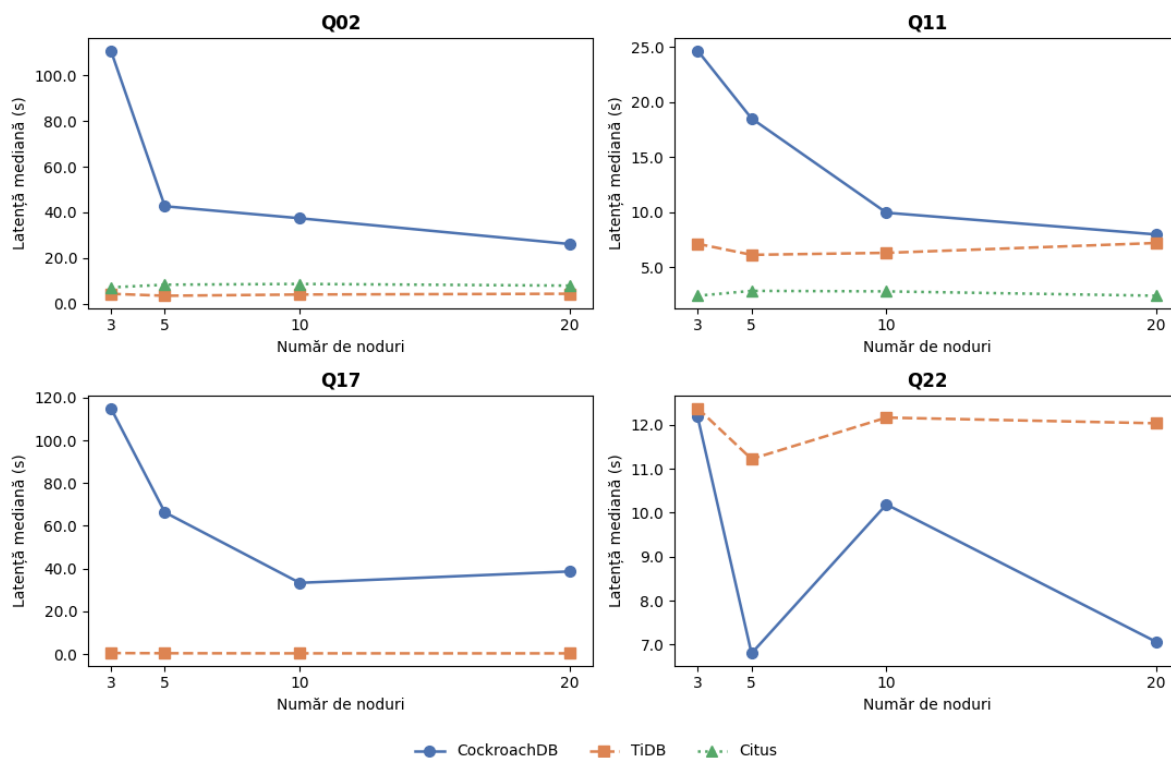


Figura 5.9: Grupa 3 - Interogări conduse de subinterogări: latența mediană în funcție de numărul de noduri.

Grupa 3 - Interogări conduse de subinterogări: Hartă termică a latenței mediane (s)

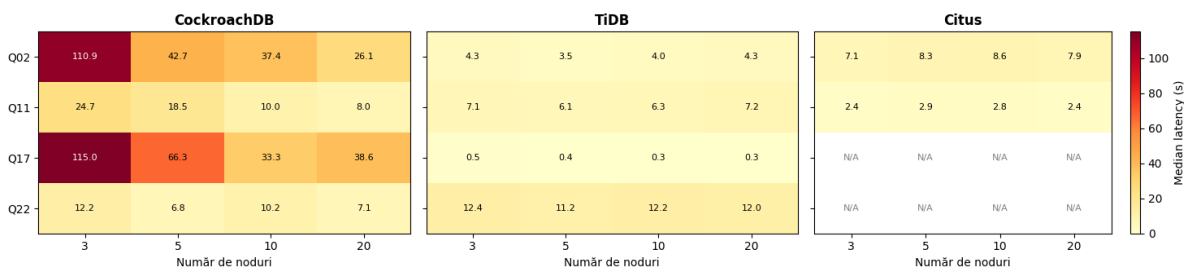


Figura 5.10: Grupa 3 - Interogări conduse de subinterogări: harta termică a latenței mediane (secunde).

Figura 5.9 și harta termică din figura 5.10 arată o performanță împărțită, fragmentată, fără un câștigător consistent: fiecare sistem domină pe câte o interogare diferită din grupă. TiDB este sistemul cu cea mai mică latență absolută pe Q2 (3, 5–4, 3 ≈ secunde la toate configurațiile) și pe Q17 (unde latența scade sub o secundă la toate configurațiile, 0, 3–0, 5 ≈ secunde). Citus este cel mai rapid pe Q11 (2, 4 – 2, 9 ≈ secunde) acolo unde poate executa interogarea, dar **nu produce rezultate pe Q17 și Q22**. CockroachDB pierde clar pe Q2 și Q17 la 3 noduri (110,9 respectiv

115,0≈secunde) dar recuperează semnificativ prin paralelism. Fragmentarea pe această grupă nu este arbitrară, ea reflectă diferențele profunde dintre strategiile fiecărui planificator de a procesa subinterogări corelate.

Grupa 3 - Interogări conduse de subinterogări: Accelație relativă la linia de bază cu 3 noduri

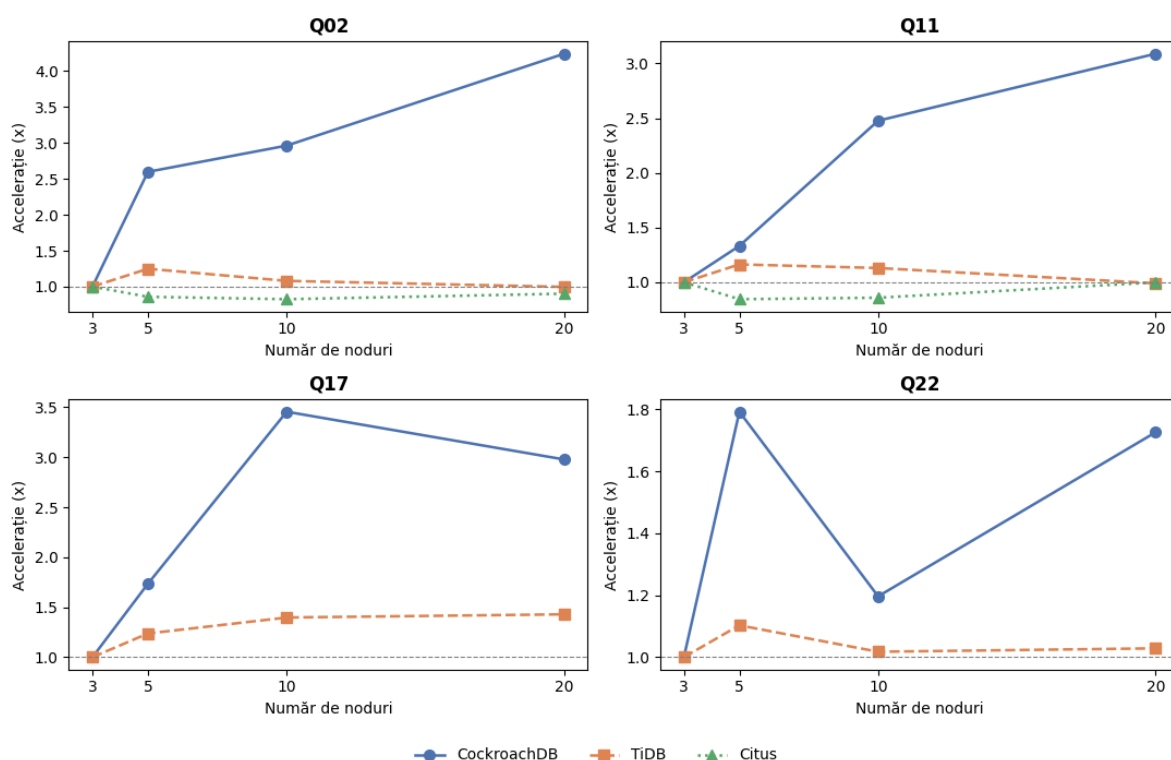


Figura 5.11: Grupa 3 - Interogări conduse de subinterogări: accelerația relativă la linia de bază cu 3 noduri.

Analiza accelerației din figura 5.11 este profund asimetrică între sisteme. CockroachDB prezintă cele mai pronunțate accelerații din grupă, atingând aproximativ $4,2\times$ pe Q2 și $3,4\times$ pe Q17 la 10 noduri (urmate de o regresie ușoară la 20 de noduri pe Q17). TiDB rămâne practic plat la toate cele patru interogări, cu accelerații sub $1,4\times$ chiar și la 20 de noduri. Citus, în mod cel mai surprinzător, prezintă accelerații nule sau ușor sub-unitare pe Q2 și Q11 (factorul $0,9-1,0\times$ la toate configurațiile), comportament care diferă radical de tiparul scalabil observat pe primele două grupe.

Grupa 3 - Interogări conduse de subinterogări: Distribuția rulărilor - 10 noduri

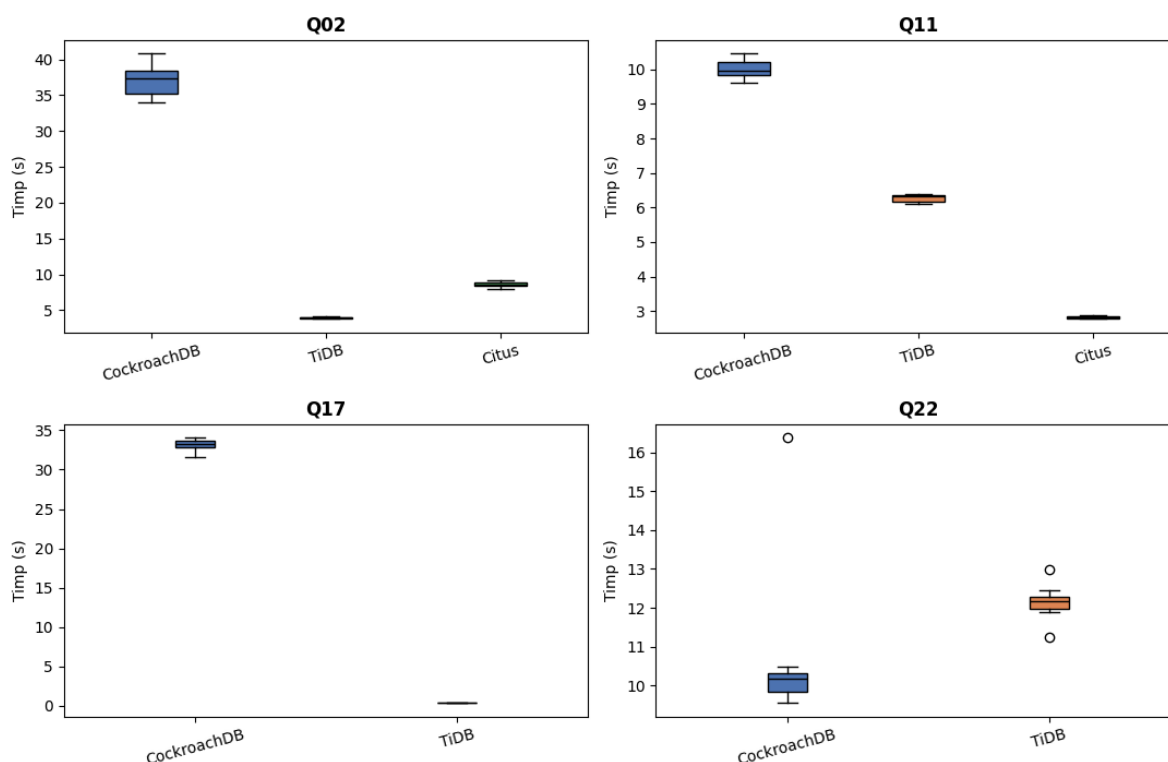


Figura 5.12: Grupa 3 - Interogări conduse de subinterogări: distribuția celor zece rulări temporizate la configurația cu 10 noduri.

Distribuția rulărilor la 10 noduri, prezentată în figura 5.12, este în general strâmtă, cu o singură excepție notabilă: CockroachDB pe Q22 prezintă o valoare aberantă clară la aproximativ 16 secunde, în timp ce restul rulărilor sunt concentrate în jurul valorii de 10 secunde. Această valoare aberantă unică explică și tiparul non-monoton observat în figura 5.9 pentru Q22 pe CockroachDB (12,2 $\rightarrow$ 6,8 $\rightarrow$ 10,2 $\rightarrow$ 7,1 secunde pe măsura creșterii numărului de noduri).

5.4 Grupa 4 - Interogări în mai multe etape

A patra grupă este cea mai complexă și diferită din studiu, formată din Q4, Q12, Q13, Q15, Q16, Q18, Q20 și Q21, cuprinzând opt interogări care combină tehnici relaționale diverse: agregări nestate, join-uri auto-referențiale, expresii de tabel comune (CTE), îmbinări laterale, precum și anti-join-uri. Q4 și Q12 efectuează agregări cu condiții de existență pe `lineitem`, Q13 calculează distribuția numărului de comenzi pe client printr-o îmbinare exterioară urmată de agregare în două etape, Q15 folosește o expresie CTE pentru a precalculea veniturile pe furnizor înainte de a identifica furnizorul cu venit maxim, Q16 calculează numărul de furnizori distincți pe combinație de proprietăți ale părților, Q18 identifică clienții cu cele mai mari comenzi printr-o îmbinare laterală, Q20 selectează furnizorii cu stocuri excedentare pentru o națiune dată, iar Q21

detectează furnizorii responsabili de livrări întârziate printr-o auto-îmbinare a tabelii lineitem cu construcții EXISTS și NOT EXISTS. Diversitatea grupei face ca observațiile generale să fie mai puțin uniforme de la o interogare la alta.

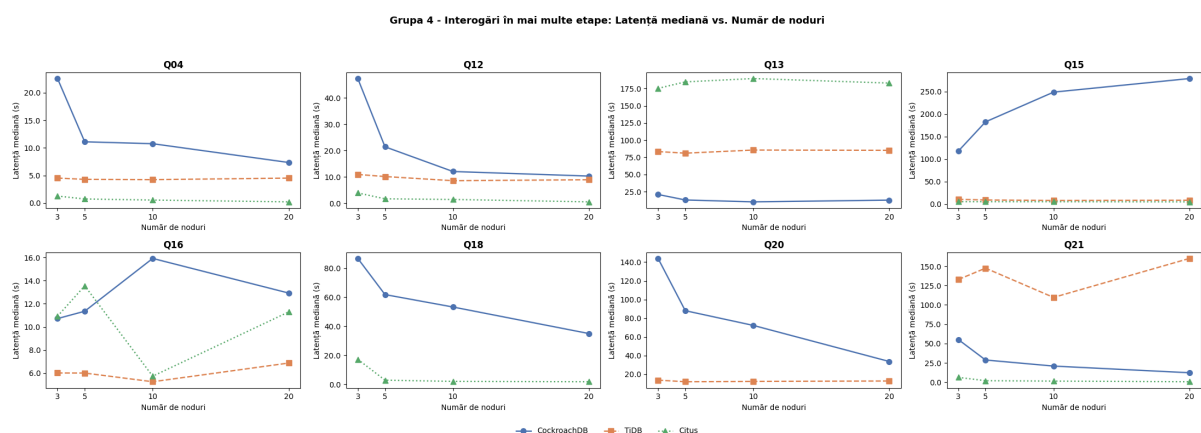


Figura 5.13: Grupa 4 - Interogări în mai multe etape: latența mediană în funcție de numărul de noduri.

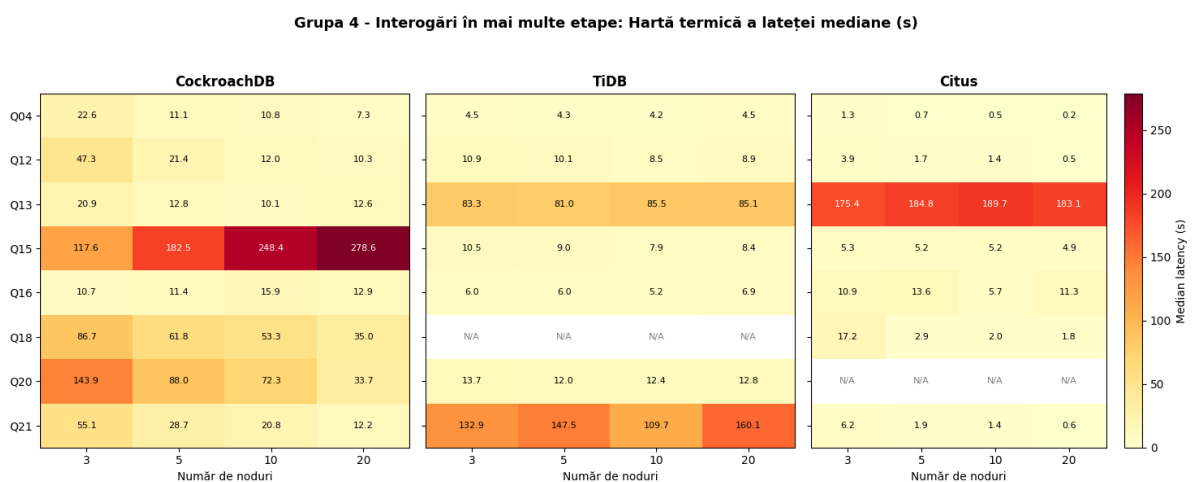


Figura 5.14: Grupa 4 - Interogări în mai multe etape: harta termică a latenței mediane (secunde).

Figura 5.13 și harta termică din figura 5.14 arată o dispersie a performanței mult mai mare decât pe grupele precedente, fără un câștigător dominant. Citus oferă din nou cea mai mică latență absolută pe Q4, Q12, Q18 și Q21 (Q21 fiind cazul cel mai spectaculos cu 0,6 secunde la 20 de noduri), dar înregistrează un eșec total pe Q13 unde latența rămâne staționară în jurul valorii de $180 \approx$ secunde indiferent de numărul de noduri. TiDB oferă rezultate competitive pe Q4, Q12 și Q16 dar prezintă două anti-tipare severe pe Q13 ($85 \approx$ de secunde la toate configurațiile) și pe Q21 ($130 - 160 \approx$ secunde, valoare neobișnuit de mare). CockroachDB prezintă cea mai gravă anti-scalare din întreg benchmarkul pe Q15, cu latența crescând de la 117,6 secunde la 3 noduri la 278,6 secunde la 20 de noduri. În rest, CockroachDB se comportă rezonabil pe Q4,

Q12, Q20 și Q21, iar rezultatul pe Q13 este în mod surprinzător mult mai bun decât Citus și TiDB.

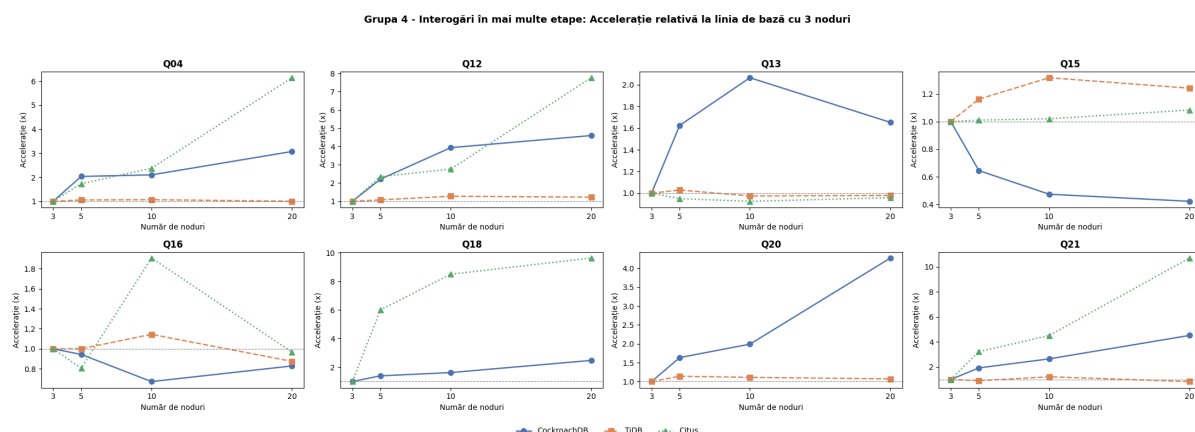


Figura 5.15: Grupa 4 - Interogări în mai multe etape: accelerația relativă la linia de bază cu 3 noduri.

Analiza accelerației din figura 5.15 amplifică contrastele observate. Citus prezintă cele mai puternice accelerații din grupă pe Q4, Q12, Q18 și Q21, atingând factori între $6\times$ și $10\times$ la 20 de noduri. CockroachDB scalează robust pe Q20 ($4,3\times$), Q21 ($4,5\times$) și Q12 ($4,6\times$), dar prezintă un caz unic de accelerație fundamental sub-unitară pe Q15, cu factor de aproximativ $0,42\times$ la 20 de noduri, adică sistemul devine cu 58% mai lent prin adăugarea de noduri. TiDB rămâne în mare parte plat pe această grupă, cu accelerații sub $1,3\times$ pe șapte din cele opt interogări, cu excepția Q4 unde nici TiDB nu beneficiază semnificativ.

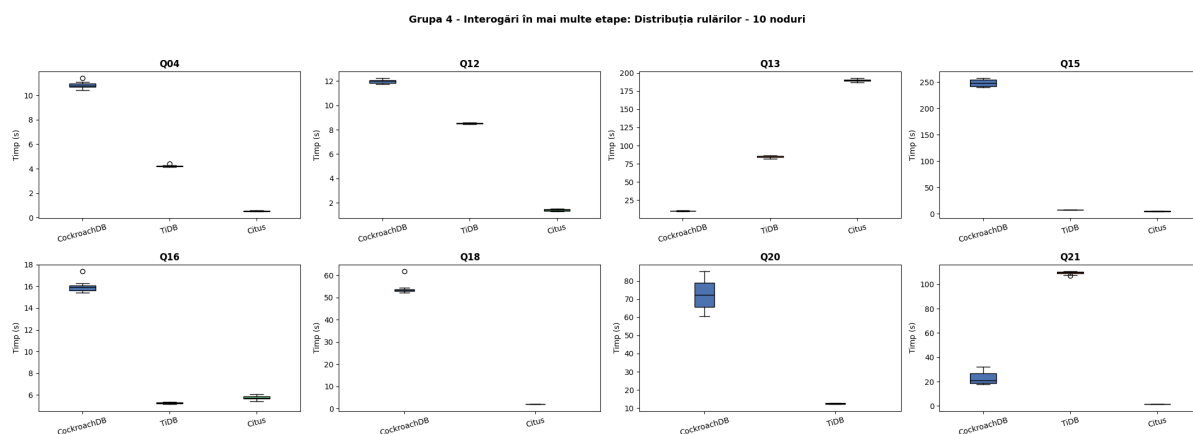


Figura 5.16: Grupa 4 - Interogări în mai multe etape: distribuția celor zece rulări temporizate la configurația cu 10 noduri.

Distribuția rulărilor la 10 noduri, prezentată în figura 5.16, indică două cazuri de variabilitate crescută care merită atenție. Q16 prezintă pentru Citus o cutie cu amplitudine mai mare decât pe celelalte sisteme, sugerând că planul de execuție al acestei interogări produce rezultate

inconsistente între rulări. Această observație este confirmată și de figura latenței (figura 5.13), care arată un tipar nelinear clar pentru Citus pe Q16 (10,9 → 13,6 → 5,7 → 11,3 ≈ secunde). Q20 pe CockroachDB prezintă cea mai largă cutie din întreg studiul, cu interval de aproximativ 20 secunde și extreme de aproximativ 60 și 80 secunde.

5.5 Sinteză intergrupală

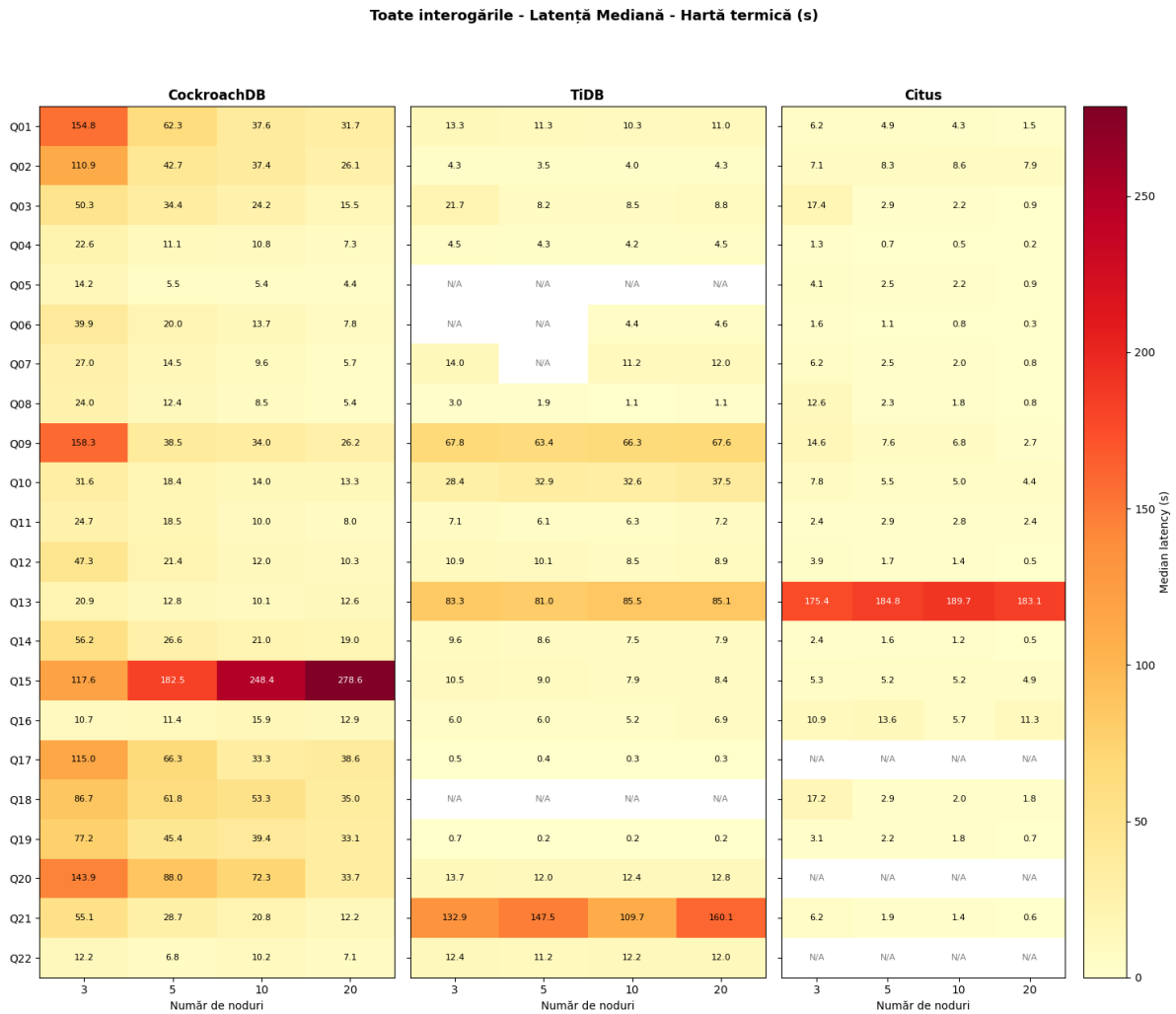


Figura 5.17: Harta termică globală a latenței mediane (secunde) pentru toate cele 22 de interogări TPC-H pe toate cele patru configurații de cluster.

Harta termică globală din figura 5.17 oferă imaginea de ansamblu cea mai compactă a rezultatelor. Prima observație este acoperirea diferită a celor trei sisteme: CockroachDB execută toate cele 22 de interogări la toate cele patru configurații, în timp ce TiDB nu produce rezultate pentru Q5 și Q18 la nicio configurație și prezintă lipsuri tranzitorii pe Q6 (configurațiile cu 3 și 5 noduri) și pe Q7 (configurația cu 5 noduri), iar Citus nu poate executa Q17, Q20 și Q22 din motive arhitecturale documentate în secțiunile precedente. Această diferență de acoperire

constituie ea însăși un rezultat: pentru medii care prioritizează compatibilitatea SQL strictă cu specificația TPC-H peste performanța de vârf, CockroachDB este singurul sistem din studiu care nu necesită refactorizări sau ajustări de configurare a schemei.

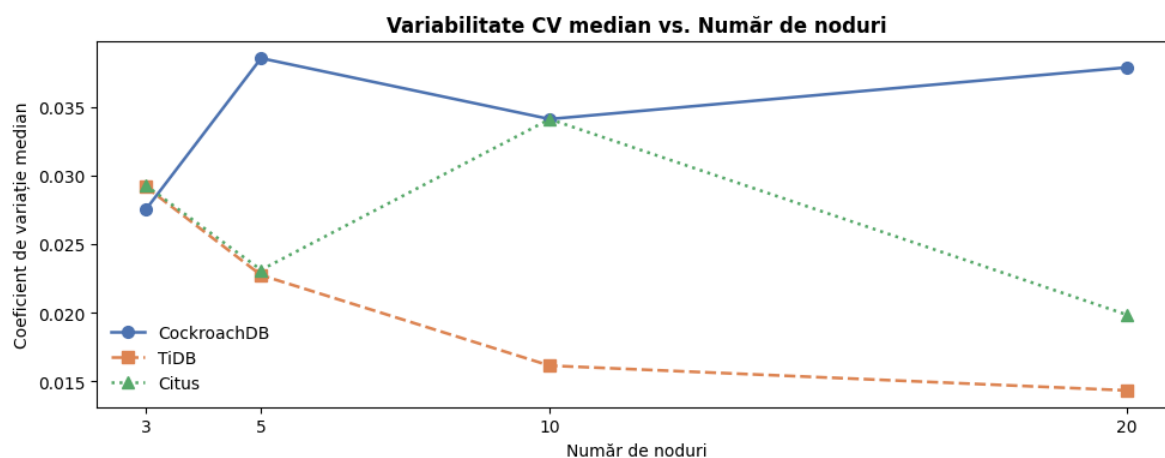


Figura 5.18: Coeficientul de variație median al rulărilor temporizate, agregat pe toate interogările executabile, în funcție de numărul de noduri.

Stabilitatea măsurătorilor între rulări, prezentată în figura 5.18 prin coeficientul de variație agregat, oferă o perspectivă complementară. TiDB prezintă cea mai mare stabilitate, cu un coeficient de variație care descrește monoton de la 0,029 la 3 noduri până la 0,014 la 20 de noduri. Citus prezintă un comportament mai zgomotos dar care se stabilizează la configurațiile mari (CV de 0,020 la 20 de noduri). CockroachDB rămâne sistemul cu cea mai mare variabilitate, oscilând în jurul valorii de 0,038.

Bază de date	media	median	min	max
Citus	5,93	5,22	0,90	18,63
CockroachDB	3,33	3,16	0,42	6,05
TiDB	1,34	1,07	0,76	3,23

Tabela 5.1: Sumar accelerație relativă la linia de bază cu 3 noduri

Capitolul 6

Analiza rezultatelor

În capitolul precedent am prezentat rezultatele și comportamentul celor trei sisteme pe fiecare grupă de interogări, identificând tipare și anomalii. Urmează să analizăm fundamentul cauzal al acestor observații prin intermediul planurilor de execuție colectate pentru fiecare interogare și fiecare configurație.

6.1 CockroachDB

Planurile DistSQL urmează un tipar uniform: optimizatorul produce un graf de fluxuri distribuit automat pe noduri, fiecare nod procesând un range distinct al tabelii. Majoritatea planurilor indică `distribution: full` și `vectorized: true`, oferind o cale predictibilă către scalabilitate prin replicarea muncii pe mai multe noduri prin paralelizare. Aspectul critic al acestei strategii este absența unei agregări parțiale la nivelul fiecărui range înainte de transferul către nodul de coordonare: planul pentru Q1 la 20 de noduri arată un scan cu spans: `FULL SCAN` pe `lineitem`, urmat de un `group (hash)` la rădăcină care procesează fluxul complet de tuple post-filtrare. Această arhitectură explică diferența de latență suplimentară pe Grupa 1 și accelerația sa pronunțată, existând o posibilitate de paralelizare a muncii, astfel fiecare nod adițional are un impact proporțional.

Cazul cel mai important pentru caracterizarea limitelor CockroachDB este anti-scalarea pe Q15. Planurile colectate la 3 și la 20 de noduri sunt identice și ambele indică `distribution: local`, în contrast cu `distribution: full` de pe restul interogărilor. La 20 de noduri, beneficiul paralelizării scanării inițiale a `lineitem` este anulat de overhead-ul de agregare a rezultatelor de la 20 de surse într-un buffer central, producând o latență de 2,4 ori mai mare decât la 3 noduri.

6.2 TiDB

Planurile TiDB expun o arhitectură fundamental diferită: stratul de stocare TiKV este accesat prin coprocesoare (`cop[tikv]`) care evaluează filtre și agregări parțiale local, iar rezultatele parțiale sunt transmise stratului SQL pentru operațiile rămase. Stratul de procesare SQL operează însă pe o singură instanță per interogare, paralelismul fiind realizat doar la nivelul accesului la stocare. Această arhitectură produce două tipare distincte de performanță. Pe Grupa 1, delegarea agregărilor la TiKV este eficientă, planul pentru Q1 la 20 de noduri împinge HashAgg integral în `cop[tikv]` cu 132 de `cop-task-uri`. În schimb, pe Grupele 2 și 4 planurile materializează îmbinări mari la stratul SQL pe o singură instanță: planul Q9 la 20 de noduri construiește tabele hash de 2,24 GB (cu revărsare 1,97 GB pe disc) pe `partsupp` și 2,52 GB pe `orders`, întregul lanț rulând pe o singură instanță TiDB.

Absențele Q5 și Q18 (OOM la stratul SQL chiar cu 24 GB memorie RAM pe nod) reflectă limitări de implementare ale stratului SQL.

6.3 Citus

Citus distribuie tabelele specificate manual pe shard-uri (sub-tabele PostgreSQL fizice) plasate pe nodurile worker, iar planificatorul transformă interogările primite de coordonator în secvențe de sub-interogări PostgreSQL trimise paralel către nodurile worker. În benchmark-ul utilizat, descris în capitolul 4, `lineitem` și `orders` sunt distribuite și co-localizate prin cheia `l_orderkey/o_orderkey`, iar restul tabelor sunt replicate ca tabele de referință. Această configurație produce trei tipare principale.

Pe Grupa 1, planul pentru Q1 generează 32 de task-uri paralele care execută local `Parallel Seq Scan` urmat de `Partial HashAggregate` pe fiecare shard, coordonatorul primind doar tuple pre-agregate.

Pe Grupa 2, planul pentru Q5 transmite fiecărui nod o interogare SQL conținând îmbinarea integrală în cinci sensuri, executată local pe shard-ul respectiv, accelerațiile aproape liniare de $19\times$ pe Q3 și $16\times$ pe Q8 reflectă această proprietate.

Pe Grupa 3 însă, când toate tabelele implicate sunt de referință (cazul Q2, Q11), planul indică `Task Count: 1`: planificatorul delegă întreaga interogare unui singur nod în loc să paralelizeze, producând performanță stabilă dar fără scalare orizontală.

Capitolul 7

Concluzii

În această lucrare am evaluat trei sisteme SQL distribuite, și anume: CockroachDB, TiDB și Citus pe suita analitică TPC-H la factor de scară 30, pe configurații de cluster cu 3, 5, 10 și 20 de noduri. Analiza celor 22 de interogări împreună cu interpretarea planurilor de execuție corespunzătoare a evidențiat trei arhitecturi distincte, fiecare cu aplicabilitatea ei.

Concluzia generală a studiului este că alegerea sistemului optim pentru o aplicație analitică distribuită depinde de combinația dintre profilul dominant al cerințelor de lucru și nivelul de acoperire SQL necesar, nu doar de performanța brută. Pentru cerințe de tip analitic OLAP și control complet asupra strategiei de partiționare Citus reprezintă alegerea cu cele mai bune rezultate. Pentru cerințe mixte sau pentru cazuri în care acoperirea SQL și predictibilitatea primează asupra latenței absolute, CockroachDB constituie o alegere echilibrată, rămânând singurul sistem care a rulat toate interogările. Pentru cerințe mixte, TiDB rămâne competitiv prin mecanismele sale de separare.

7.1 Direcții viitoare de cercetare

Sunt mai multe direcții de continuare a lucrării. O primă direcție este reevaluarea TiDB prin utilizarea componentei TiFlash, un motor columnar replicat care nu a făcut parte din configurația acestui studiu dar care ar fi putut modifica semnificativ rezultate. O altă direcție de dezvoltare al acestui studiu este extinderea evaluării pe suita TPC-C, care ar permite caracterizarea sistemelor pe profil tranzacțional OLTP și compararea echilibrului, aceste sisteme fiind proiectate în principal pentru acest tip de procesare. O a treia direcție este compararea cu sisteme analitice native columnare. O a patra direcție ar fi efectul indecșilor creați manuali fie la recomandarea motoarelor, fie prin analiza manuală, sistemul CRDB sugerând explicit indecși pentru fiecare interogare. O direcție complementară este evaluarea sub concurență, prin execuția mai multor interogări simultan. O altă direcție firească este creșterea factorului de scară pentru a evalua comportamentul sistemelor la volume mai mari.

Anexa A

Schemă și configurație de partiționare

A.1 Schema comună TPC-H

Instrucțiunile de mai jos reprezintă schema ca executată pe CockroachDB și TiDB. Varianta Citus este identică, cu excepția că toate clauzele REFERENCES și FOREIGN KEY sunt eliminate.

Cod A.1: Schema TPC-H - Comun CockroachDB și TiDB

```
CREATE TABLE region
(
    r_regionkey INT PRIMARY KEY,
    r_name      CHAR(25) NOT NULL,
    r_comment   VARCHAR(152)
);

CREATE TABLE nation
(
    n_nationkey INT PRIMARY KEY,
    n_name      CHAR(25) NOT NULL,
    n_regionkey INT NOT NULL REFERENCES region (r_regionkey),
    n_comment   VARCHAR(152)
);

CREATE TABLE supplier
(
    s_suppkey   INT PRIMARY KEY,
    s_name      CHAR(25) NOT NULL,
    s_address   VARCHAR(40) NOT NULL,
    s_nationkey INT NOT NULL REFERENCES nation (n_nationkey),
    s_phone     CHAR(15) NOT NULL,
    s_acctbal   DECIMAL(15, 2) NOT NULL,
    s_comment   VARCHAR(101) NOT NULL
);

CREATE TABLE part
```

```

(
    p_partkey      INT PRIMARY KEY,
    p_name         VARCHAR(55)      NOT NULL,
    p_mfgr         CHAR(25)         NOT NULL,
    p_brand        CHAR(10)         NOT NULL,
    p_type         VARCHAR(25)      NOT NULL,
    p_size         INT              NOT NULL,
    p_container    CHAR(10)         NOT NULL,
    p_retailprice  DECIMAL(15, 2)   NOT NULL,
    p_comment      VARCHAR(23)      NOT NULL
);

CREATE TABLE customer
(
    c_custkey      INT PRIMARY KEY,
    c_name         VARCHAR(25)      NOT NULL,
    c_address      VARCHAR(40)      NOT NULL,
    c_nationkey    INT              NOT NULL REFERENCES nation (n_nationkey),
    c_phone        CHAR(15)         NOT NULL,
    c_acctbal      DECIMAL(15, 2)   NOT NULL,
    c_mktsegment   CHAR(10)         NOT NULL,
    c_comment      VARCHAR(117)     NOT NULL
);

CREATE TABLE partsupp
(
    ps_partkey     INT              NOT NULL REFERENCES part (p_partkey),
    ps_suppkey     INT              NOT NULL REFERENCES supplier (s_suppkey),
    ps_availqty    INT              NOT NULL,
    ps_supplycost  DECIMAL(15, 2)   NOT NULL,
    ps_comment     VARCHAR(199)     NOT NULL,
    PRIMARY KEY (ps_partkey, ps_suppkey)
);

CREATE TABLE orders
(
    o_orderkey     INT PRIMARY KEY,
    o_custkey      INT              NOT NULL REFERENCES customer (c_custkey),
    o_orderstatus  CHAR(1)          NOT NULL,
    o_totalprice   DECIMAL(15, 2)   NOT NULL,
    o_orderdate    DATE             NOT NULL,
    o_orderpriority CHAR(15)         NOT NULL,
    o_clerk        CHAR(15)         NOT NULL,
    o_shippriority INT              NOT NULL,
    o_comment      VARCHAR(79)      NOT NULL
);

CREATE TABLE lineitem
(
    l_orderkey     INT              NOT NULL REFERENCES orders (o_orderkey),
    l_partkey      INT              NOT NULL REFERENCES part (p_partkey),

```

```

l_suppkey      INT          NOT NULL REFERENCES supplier (s_suppkey),
l_linenumber   INT          NOT NULL,
l_quantity     DECIMAL(15, 2) NOT NULL,
l_extendedprice DECIMAL(15, 2) NOT NULL,
l_discount     DECIMAL(15, 2) NOT NULL,
l_tax         DECIMAL(15, 2) NOT NULL,
l_returnflag   CHAR(1)      NOT NULL,
l_linestatus   CHAR(1)      NOT NULL,
l_shipdate     DATE          NOT NULL,
l_commitdate   DATE          NOT NULL,
l_receiptdate  DATE          NOT NULL,
l_shipinstruct CHAR(25)     NOT NULL,
l_shipmode     CHAR(10)     NOT NULL,
l_comment      VARCHAR(44)   NOT NULL,
PRIMARY KEY (l_orderkey, l_linenumber),
FOREIGN KEY (l_partkey, l_suppkey) REFERENCES
    partsupp (ps_partkey, ps_suppkey)
);

```

A.2 Configurația de distribuție Citus

Cod A.2: Configurația de distribuție Citus

```

SELECT create_reference_table('region');
SELECT create_reference_table('nation');
SELECT create_reference_table('supplier');
SELECT create_reference_table('part');
SELECT create_reference_table('partsupp');
SELECT create_reference_table('customer');
SELECT create_distributed_table('orders', 'o_orderkey');
SELECT create_distributed_table('lineitem',
    'l_orderkey', colocate_with => 'orders');

```


Anexa B

Tabel mediane

Tabela B.1: Query execution times (seconds) by system and node count

Query	CockroachDB				TiDB				Citrus			
	3	5	10	20	3	5	10	20	3	5	10	20
Q01	154.80	62.26	37.63	31.75	13.28	11.33	10.34	11.01	6.23	4.87	4.28	1.51
Q02	110.89	42.69	37.43	26.14	4.33	3.47	4.01	4.35	7.13	8.31	8.65	7.91
Q03	50.33	34.37	24.21	15.52	21.67	8.16	8.47	8.83	17.43	2.94	2.18	0.94
Q04	22.58	11.10	10.76	7.35	4.52	4.29	4.24	4.52	1.26	0.73	0.53	0.21
Q05	14.17	5.49	5.40	4.39	–	–	–	–	4.13	2.45	2.25	0.94
Q06	39.94	20.00	13.67	7.83	–	–	4.37	4.58	1.58	1.07	0.75	0.30
Q07	27.01	14.54	9.61	5.70	14.02	–	11.23	11.96	6.16	2.51	1.96	0.78
Q08	24.00	12.39	8.47	5.38	3.03	1.87	1.06	1.13	12.56	2.32	1.80	0.78
Q09	158.27	38.48	34.02	26.17	67.77	63.36	66.34	67.58	14.61	7.57	6.76	2.67
Q10	31.62	18.37	14.00	13.29	28.44	32.90	32.63	37.51	7.75	5.46	5.00	4.41
Q11	24.68	18.52	9.96	7.99	7.14	6.14	6.32	7.21	2.43	2.87	2.82	2.43
Q12	47.28	21.39	12.02	10.28	10.89	10.11	8.53	8.89	3.88	1.65	1.41	0.50
Q13	20.85	12.83	10.10	12.62	83.32	81.00	85.55	85.13	175.40	184.83	189.69	183.10
Q14	56.21	26.62	21.05	19.00	9.56	8.59	7.45	7.94	2.37	1.62	1.17	0.45
Q15	117.61	182.50	248.45	278.64	10.47	9.01	7.95	8.43	5.27	5.22	5.17	4.86
Q16	10.72	11.36	15.93	12.93	6.01	6.00	5.25	6.87	10.93	13.56	5.73	11.30
Q17	115.02	66.30	33.30	38.64	0.46	0.37	0.33	0.32	–	–	–	–
Q18	86.74	61.84	53.27	34.98	–	–	–	–	17.24	2.87	2.03	1.79
Q19	77.22	45.43	39.35	33.13	0.74	0.22	0.21	0.23	3.07	2.20	1.77	0.70
Q20	143.95	88.04	72.30	33.69	13.74	12.03	12.36	12.82	–	–	–	–
Q21	55.12	28.70	20.84	12.20	132.86	147.54	109.70	160.14	6.22	1.93	1.38	0.58
Q22	12.19	6.80	10.19	7.06	12.38	11.22	12.17	12.04	–	–	–	–

Anexa C

Mesaje eroare

Cod C.1: Citus – eroare Join distribuit

```
ERROR: complex joins are only supported when all distributed tables are
co-located and joined on their distribution columns
```

Cod C.2: TiDB – eroare Q5

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your TiDB version for the right syntax to use near
'...' at line 1
```

Cod C.3: Citus – OOM killer, nod worker, Q18, 16 GB RAM

```
[ 2746.401130] oom-kill:constraint=CONSTRAINT_NONE,nodemask=(null),
cpuset=/,mems_allowed=0,global_oom,
task_memcg=/system.slice/system-postgresql.slice/
postgresql@17-main.service,
task=postgres,pid=18225,uid=114
[ 2746.401143] Out of memory: Killed process 18225 (postgres)
total-vm:8719596kB, anon-rss:7499020kB,
file-rss:692kB, shmem-rss:14264kB, UID:114
pgtables:14912kB oom_score_adj:0
```

Cod C.4: TiDB – eroare SQL quota Q18

```
ERROR 8175 (HY000): Your query has been cancelled due to exceeding the
allowed memory limit for a single SQL query. Please try narrowing your
query scope or increase the tidb_mem_quota_query limit and try again.
[conn=354419924]
```

Cod C.5: TiDB – OOM killer, nod SQL, Q18, 16 GB RAM

```
[ 7134.599098] tidb-server invoked oom-killer:
gfp_mask=0x1100cca(GFP_HIGHUSER_MOVABLE), order=0, oom_score_adj=0
[ 7134.599375] oom-kill:constraint=CONSTRAINT_NONE,nodemask=(null),
```

```
cpuset=/,mems_allowed=0,global_oom,  
task_memcg=/system.slice/tidb-4000.service,  
task=tidb-server,pid=10752,uid=0  
[ 7134.599417] Out of memory: Killed process 10752 (tidb-server)  
total-vm:8157780kB, anon-rss:5510896kB,  
file-rss:0kB, shmem-rss:0kB, UID:0  
pgtables:12372kB oom_score_adj:0
```

Bibliografie

- [1] Transaction Processing Performance Council, “TPC-H benchmark specification,” <https://www.tpc.org/tpch/>.
- [2] M. Stonebraker, “The case for shared nothing,” *IEEE Database Engineering Bulletin*, vol. 9, no. 1, 1986.
- [3] D. DeWitt and J. Gray, “Parallel database systems: the future of high performance database systems,” *Communications of the ACM*, vol. 35, no. 6, p. 85–98, 1992. [Online]. Available: <https://dl.acm.org/doi/10.1145/129888.129894>
- [4] B. Stopford, “Shared nothing v.s. shared disk architectures: An independent view,” Nov 2009, accesat: 2026. [Online]. Available: <https://benstopford.com/2009/11/24/understanding-the-shared-nothing-architecture/>
- [5] E. A. Brewer, “Towards robust distributed systems,” p. 7, july 2000. [Online]. Available: <https://dl.acm.org/doi/10.1145/343477.343502>
- [6] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, p. 51–59, june 2002. [Online]. Available: <https://dl.acm.org/doi/10.1145/564585.564601>
- [7] E. Brewer, “CAP twelve years later: How the ”rules” have changed,” *Computer*, vol. 45, no. 2, pp. 23–29, feb 2012. [Online]. Available: <https://ieeexplore.ieee.org/document/6133253>
- [8] M. Kleppmann, “A critique of the cap theorem,” 2015. [Online]. Available: <https://arxiv.org/abs/1509.05393>
- [9] D. Abadi, “Consistency tradeoffs in modern distributed database system design: Cap is only part of the story,” *Computer*, vol. 45, no. 2, p. 37–42, Feb. 2012. [Online]. Available: <https://doi.org/10.1109/MC.2012.33>
- [10] M. P. Herlihy and J. M. Wing, “Linearizability: a correctness condition for concurrent objects,” *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 3, p. 463–492, Jul. 1990. [Online]. Available: <https://doi.org/10.1145/78969.78972>

- [11] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Commun. ACM*, vol. 21, no. 7, p. 558–565, Jul. 1978. [Online]. Available: <https://doi.org/10.1145/359545.359563>
- [12] W. Vogels, “Eventually consistent,” *Commun. ACM*, vol. 52, no. 1, p. 40–44, Jan. 2009. [Online]. Available: <https://doi.org/10.1145/1435417.1435432>
- [13] J. Gray, P. Helland, P. O’Neil, and D. Shasha, “The dangers of replication and a solution,” in *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD ’96. New York, NY, USA: Association for Computing Machinery, 1996, p. 173–182. [Online]. Available: <https://doi.org/10.1145/233269.233330>
- [14] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, ser. USENIX ATC’14. USA: USENIX Association, 2014, p. 305–320.
- [15] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *J. ACM*, vol. 32, no. 2, p. 374–382, Apr. 1985. [Online]. Available: <https://doi.org/10.1145/3149.214121>
- [16] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: a tutorial,” *ACM Comput. Surv.*, vol. 22, no. 4, p. 299–319, Dec. 1990. [Online]. Available: <https://doi.org/10.1145/98163.98167>
- [17] L. Lamport, “The part-time parliament,” *ACM Trans. Comput. Syst.*, vol. 16, no. 2, p. 133–169, May 1998. [Online]. Available: <https://doi.org/10.1145/279227.279229>
- [18] —, “Paxos made simple,” *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001), pp. 51–58, December 2001. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/>
- [19] T. D. Chandra, R. Griesemer, and J. Redstone, “Paxos made live: an engineering perspective,” in *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing*, ser. PODC ’07. New York, NY, USA: Association for Computing Machinery, 2007, p. 398–407. [Online]. Available: <https://doi.org/10.1145/1281100.1281103>
- [20] H. Howard and R. Mortier, “Paxos vs raft: have we reached consensus on distributed consensus?” in *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data*, ser. PaPoC ’20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: <https://doi.org/10.1145/3380787.3393681>
- [21] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proceedings of the 1979*

- ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '79. New York, NY, USA: Association for Computing Machinery, 1979, p. 23–34. [Online]. Available: <https://doi.org/10.1145/582095.582099>
- [22] G. Graefe, “Query evaluation techniques for large databases,” *ACM Comput. Surv.*, vol. 25, no. 2, p. 73–169, Jun. 1993. [Online]. Available: <https://doi.org/10.1145/152610.152611>
- [23] —, “The cascades framework for query optimization,” *IEEE Data Eng. Bull.*, vol. 18, no. 3, pp. 19–29, 1995.
- [24] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, “How good are query optimizers, really?” *Proc. VLDB Endow.*, vol. 9, no. 3, p. 204–215, Nov. 2015. [Online]. Available: <https://doi.org/10.14778/2850583.2850594>
- [25] D. Kossmann, “The state of the art in distributed query processing,” *ACM Comput. Surv.*, vol. 32, no. 4, p. 422–469, Dec. 2000. [Online]. Available: <https://doi.org/10.1145/371578.371598>
- [26] M. T. Özsu and P. Valduriez, *Principles of distributed database systems*, 4th ed. Cham, Switzerland: Springer Nature, Dec. 2019.
- [27] S. Chaudhuri, “An overview of query optimization in relational systems,” in *Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems*, ser. PODS '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 34–43. [Online]. Available: <https://doi.org/10.1145/275487.275492>
- [28] J. M. Hellerstein, “Optimization techniques for queries with expensive methods,” *ACM Trans. Database Syst.*, vol. 23, no. 2, p. 113–157, Jun. 1998. [Online]. Available: <https://doi.org/10.1145/292481.277627>
- [29] W. P. Yan and P.-r. Larson, “Eager aggregation and lazy aggregation,” in *Proceedings of the 21th International Conference on Very Large Data Bases*, ser. VLDB '95. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1995, p. 345–357.
- [30] M. Kleppmann, *Designing data-intensive applications*. Sebastopol, CA: O'Reilly Media, Mar. 2017.
- [31] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “Cockroachdb: The resilient geo-distributed sql database,” in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1493–1509. [Online]. Available: <https://doi.org/10.1145/3318464.3386134>

- [32] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang, W. Wei, C. Liu, J. Zhang, J. Li, X. Wu, L. Song, R. Sun, S. Yu, L. Zhao, N. Cameron, L. Pei, and X. Tang, “Tidb: a raft-based htap database,” *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3072–3084, Aug. 2020. [Online]. Available: <https://doi.org/10.14778/3415478.3415535>
- [33] U. Cubukcu, O. Erdogan, S. Pathak, S. Sannakkayala, and M. Slot, “Citrus: Distributed postgresql for data-intensive applications,” in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2490–2502. [Online]. Available: <https://doi.org/10.1145/3448016.3457551>
- [34] Cockroach Labs, “Architecture Overview — cockroachlabs.com,” <https://www.cockroachlabs.com/docs/v26.1/architecture/overview>, 2026, [Accessed: 04-05-2026].
- [35] —, “Cost-based optimizer — cockroachlabs.com,” <https://www.cockroachlabs.com/docs/v26.1/cost-based-optimizer>, 2026, [Accessed: 04-05-2026].
- [36] C. andreimatei, knz, RaduBerinde, “Distributed SQL in CockroachDB,” CockroachDB, Tech. Rep., Apr. 2016, cockroachDB RFC 20160421, Accessed: 2026-05-04. [Online]. Available: https://github.com/cockroachdb/cockroach/blob/master/docs/RFCs/20160421_distributed_sql.md
- [37] Cockroach Labs, “Join expressions — cockroachlabs.com,” <https://www.cockroachlabs.com/docs/v26.1/joins>, 2026, [Accessed: 04-05-2026].
- [38] PingCAP, “Statistics — TiDB documentation,” May 2026, accesat: 2026-05-04. [Online]. Available: <https://docs.pingcap.com/tidb/v8.5/statistics/>
- [39] —, “Explain statements that use joins — TiDB documentation,” May 2026, accesat: 2026-05-04. [Online]. Available: <https://docs.pingcap.com/tidb/v8.5/explain-joins/>
- [40] Craig Kerstiens, “How citrus works,” Sep. 2017, accesat: 2026-05-04. [Online]. Available: <https://www.citusdata.com/blog/2017/09/15/how-citus-works/>
- [41] Citus Data, “Choosing distribution column,” accesat: 2026-05-04. [Online]. Available: https://docs.citusdata.com/en/v13.0/sharding/data_modeling.html#choosing-distribution-column